Learning Hub Master Series • Technical Handbook

C PROGRAMMING FOR LEARNING HUB

คู่มือการเขียนโปรแกรมภาษา C ฉบับวิศวกรรมและการอ้างอิงระดับมืออาชีพ

การสังเคราะห์เนื้อหาหลักสูตรระดับมหาวิทยาลัยอย่างเข้มข้นและครบวงจร (สัปดาห์ที่ 1 ถึง 12) ผสานกับหัวข้อระบบระดับลึกเชิงวิศวกรรม: ความปลอดภัยของหน่วยความจำ, กลไกของพอยน์เตอร์, ฟังก์ชันพอยน์เตอร์, สถาปัตยกรรมระดับบิต (Bitwise), การจัดสรรหน่วยความจำแบบไดนามิกบนฮีป และการป้องกันพฤติกรรมที่ไม่พึงประสงค์ (Undefined Behavior)

- รากฐานระบบ & IPO
- อาร์เรย์หลายมิติ
- พอยน์เตอร์ & พอยน์เตอร์แมท
- สตรักต์ & ยูเนียน
- การจัดการไฟล์ & ไบนารี I/O
- การจัดสรรหน่วยความจำไดนามิก
- ฟังก์ชันพอยน์เตอร์
- Safe Systems C

ฉบับสังเคราะห์หลักสูตรวิชาการและตำรามาตรฐานสากล (Academic Syllabus & Textbook Synthesis Edition)
แหล่งอ้างอิงหลักสูตร: *Computer Programming (KMUTNB)* & *C Notes for Professionals (GoalKicker)*
มาตรฐานรองรับ: ISO/IEC 9899 (C99 / C11 / C17 / C23) • จัดทำขึ้นสำหรับ CLI_WS Learning Hub

สารบัญ (Table of Contents)

ส่วนที่ 1: รากฐานระบบ & การแก้ปัญหาเชิงขั้นตอนวิธี (Core Foundations & Problem Solving)

บทที่ 1: สถาปัตยกรรมคอมพิวเตอร์, การคอมไพล์ & สตรีม I/O ขั้นพื้นฐาน — ลำดับชั้นฮาร์ดแวร์, แบบจำลอง IPO, ผังงานโฟลว์ชาร์ต, ลำดับการบิลด์ 4 ขั้น, printf & scanf

บทที่ 2: ชนิดข้อมูล, ค่าลิเทอรัล, นิพจน์ & ตัวดำเนินการ — ข้อมูลพื้นฐาน, sizeof, stdint.h, การหารตัดเศษ, prefix/postfix, การแปลงชนิดข้อมูล

บทที่ 3: การตัดสินใจแยกสาขา, ตรรกศาสตร์ & การควบคุมเงื่อนไข — ความหมายเชิงตรรกะ, ตัวดำเนินการตรรกะ, การลัดวงจร Short-Circuit, if-else, switch

บทที่ 4: การวนซ้ำ, ลูป & กลไกการทำซ้ำ — Pre-test เทียบกับ Post-test, for/while/do-while, break/continue, ลูปซ้อนลูป

ส่วนที่ 2: คอลเลกชันข้อมูล & การประมวลผลข้อความ (Data Collections & Text Processing)

บทที่ 5: อาร์เรย์หนึ่งมิติ & ขั้นตอนวิธีพื้นฐาน — หน่วยความจำต่อเนื่อง, ดัชนี, ขอบเขตความปลอดภัย, การค้นหาเชิงเส้น, การกลับด้านอาร์เรย์

บทที่ 6: อาร์เรย์หลายมิติ & การคำนวณเมทริกซ์ — การจัดเรียงแบบ Row-Major, การท่องอาเรย์ 2 มิติ, การคูณเมทริกซ์, แคชประสิทธิภาพสูง

บทที่ 7: สตริง, การประมวลผลข้อความ & ไลบรารี <string.h> — อักขระ Null Terminator, ภัยบัฟเฟอร์โอเวอร์โฟลว์, fgets ปลอดภัย, string.h, ctype.h

ส่วนที่ 3: แอดเดรสหน่วยความจำ & ฟังก์ชันแบบโมดูลาร์ (Memory Addressing & Modular Functions)

บทที่ 8: พอยน์เตอร์, โมเดลหน่วยความจำ & พอยน์เตอร์อาริทเมติก — Address-of (&), Dereference (*), ความปลอดภัย NULL, ทฤษฎี Array Decay, การบวกพอยน์เตอร์

บทที่ 9: การเขียนโปรแกรมแบบโมดูลาร์, สโคป & ฟังก์ชัน — โพรโทไทป์, การส่งผ่านค่าแบบ Pass-by-Value, ขอบเขตและอายุตัวแปร auto/static/extern

บทที่ 10: จำลองการส่งผ่านแอดเดรส, อาร์เรย์ในฟังก์ชัน & ฟังก์ชันพอยน์เตอร์ — จำลอง Pass-by-Reference, อาร์เรย์ลดรูป (Decay), ฟังก์ชันพอยน์เตอร์และคอลแบ็ก

ส่วนที่ 4: ชนิดข้อมูลที่ผู้ใช้กำหนด & สตรีม I/O (User-Defined Types & Stream I/O)

บทที่ 11: อีนัม, Typedef, โครงสร้างข้อมูลสตรักต์ & การจัดเรียง Alignment — Enumerations, typedef, สตรักต์เชิงประกอบ, ตัวดำเนินการจุดและลูกศร, ไบต์แพดดิง

บทที่ 12: ยูเนียน, การจัดการระดับบิต (Bitwise) & บิตฟิลด์ — ยูเนียนแชร์หน่วยความจำ, ตัวดำเนินการบิตไวส์ (&, |, ^, ~, <<, >>), บิตฟิลด์ฮาร์ดแวร์

บทที่ 13: การจัดการไฟล์, สตรีม I/O & การทำงานกับไฟล์ไบนารี — สตรีม FILE, โหมด fopen, ข้อความเทียบกับไบนารี fread/fwrite, การสุ่มตำแหน่ง fseek

ส่วนที่ 5: การโปรแกรมระบบระดับมืออาชีพ (Professional Systems Programming)

บทที่ 14: การจัดสรรหน่วยความจำพลวัตบนฮีป & ความปลอดภัย — malloc, calloc, realloc, free, เมมโมรีลีค, แดงลิงพอยน์เตอร์, อิมพลีเมนต์เวกเตอร์ไดนามิก

บทที่ 15: พรีโปรเซสเซอร์, สุขอนามัยของมาโคร & สถาปัตยกรรมเฮดเดอร์ — ไดเรกทิฟ, กฎใส่วงเล็บป้องกันบั๊ก, การแปลงเป็นสตริง (#), การเชื่อมโทเค็น (##), อินคลูดการ์ด

บทที่ 16: อาร์กิวเมนต์ CLI, ไลบรารีมาตรฐาน & การดักจับข้อผิดพลาด — argc/argv, ฟังก์ชันแปลง strtol ปลอดภัย, ตัวแปร errno, perror, assert.h, qsort

บทที่ 17: พฤติกรรมที่ไม่พึงประสงค์ (UB), การเขียนโค้ดเชิงป้องกัน & กับดักข้อผิดพลาด — การจำแนก UB, จุดลำดับ Sequence Points, Off-by-one, แพทเทิร์นป้องกันระบบล่ม

ส่วนที่ 6: ภาคผนวกอ้างอิง (Reference Appendices)

ภาคผนวก A: ตารางลำดับความสำคัญและการจัดกลุ่มตัวดำเนินการ 15 ระดับ (Operator Precedence Table)

ภาคผนวก B: ตารางอ้างอิงรหัสจัดรูปแบบข้อมูลมาตรฐาน (Master Format Specifier Reference Table)

ภาคผนวก C: รหัสหลีกมาตรฐาน (Escape Sequences) & เส้นทางวิวัฒนาการของมาตรฐานภาษา C

บทที่ 1

สถาปัตยกรรมคอมพิวเตอร์, การคอมไพล์ & สตรีม I/O ขั้นพื้นฐาน

1.1 กายวิภาคของระบบคอมพิวเตอร์และสถาปัตยกรรมหน่วยความจำ

ระบบคอมพิวเตอร์ยุคใหม่ประกอบด้วย 4 ระบบย่อยรากฐาน: หน่วยประมวลผลกลาง (CPU), หน่วยความจำหลัก (RAM/ROM), หน่วยบันทึกข้อมูลทุติยภูมิ (SSDs/HDDs), และอุปกรณ์รอบข้างรับเข้า/ส่งออก (I/O Peripherals) โดย CPU จะทำหน้าที่ดึงและประมวลผลคำสั่งเครื่อง (Machine Instructions) ทีละคำสั่งจากหน่วยความจำหลักอย่างเป็นลำดับผ่านระบบบัส

หน่วยความจำถูกจัดเรียงเป็นช่องเซลล์ขนาด 1 ไบต์ที่มีแอดเดรสระบุตำแหน่งเรียงกันเป็นเส้นตรง แต่ละไบต์จะมีแอดเดรสตัวเลขเฉพาะตัว ซึ่งตามแบบแผนนิยมจะแสดงผลในรูปของเลขฐานสิบหก (Hexadecimal เช่น `0x7ffeefbff5ac`) ในภาษา C ตัวแปรคือชื่อเชิงสัญลักษณ์ที่คอมไพเลอร์แมปเข้ากับตำแหน่งแอดเดรสบนหน่วยความจำจริงที่เก็บรูปแบบบิตไบนารี

ระบบย่อย	หน้าที่การทำงานหลัก	ความเร็ว & ความคงทนข้อมูล	การปฏิสัมพันธ์ในภาษา C
รีจิสเตอร์ (Registers)	หน่วยจัดเก็บข้อมูลภายใน CPU ความเร็วสูงสุดสำหรับพักข้อมูลคำนวณ	< 1 ns, ข้อมูลหายเมื่อดับไฟ (Volatile)	คอมไพเลอร์จัดการอัตโนมัติ; มีคีย์เวิร์ดแนะนำ `register`
หน่วยความจำหลัก (RAM)	เก็บชุดคำสั่งโปรแกรมที่กำลังทำงาน, Stack Frames และข้อมูลบนฮีป	~10-50 ns, ข้อมูลหายเมื่อดับไฟ (Volatile)	เข้าถึงโดยตรงผ่านตัวแปรและพอยน์เตอร์
หน่วยบันทึกทุติยภูมิ	แหล่งจัดเก็บไฟล์และข้อมูลถาวร (SSDs, Hard Disks)	~10 µs - 10 ms, ข้อมูลคงทนถาวร (Non-volatile)	เข้าถึงผ่าน API สตรีมไฟล์มาตรฐาน (`fopen`, `fread`)
อุปกรณ์รอบข้าง I/O	ส่วนติดต่อกับมนุษย์ (คีย์บอร์ด, หน้าจอ) และการสื่อสารผ่านเครือข่าย	ความเร็วแปรผัน, ควบคุมด้วยอินเทอร์รัปต์	สร้างแบบจำลองผ่านสตรีมมาตรฐาน: `stdin`, `stdout`, `stderr`

1.2 ลำดับชั้นภาษาโปรแกรมและ 4 ขั้นตอนการคอมไพล์ภาษา C

ภาษาโปรแกรมมีพัฒนาการตั้งแต่ระดับต่ำ (ภาษาเครื่องที่เป็นเลขฐานสอง `0` และ `1`, ภาษาแอสเซมบลีที่ผูกติดกับสถาปัตยกรรมชิปประมวลผล) จนถึงภาษาระดับสูงอย่างภาษา C ซึ่งมอบการเขียนที่มนุษย์เข้าใจได้ มีโครงสร้างควบคุมการทำงานเป็นระบบ และมีชนิดข้อมูลที่พกพาข้ามแพลตฟอร์มได้

คอมไพเลอร์ของภาษา C ไม่ได้แปลงซอร์สโค้ดให้กลายเป็นโปรแกรมที่รันได้ในขั้นตอนเดียว แต่จะทำงานผ่าน 4 กระบวนการอย่างเป็นระบบและตรวจสอบได้:

1. **การประมวลผลขั้นต้น (Preprocessing: `cpp`)**: ประมวลผลคำสั่งพรีโปรเซสเซอร์ที่ขึ้นต้นด้วย `#` โดยตัดคอมเมนต์ทิ้ง ขยายคำสั่งมาโคร (`#define`) และแทรกเนื้อหาของไฟล์เฮดเดอร์ (`#include`) เพื่อสร้าง Translation Unit ฉบับสมบูรณ์ (`.i`)
2. **การคอมไพล์ (Compilation: `gcc -S`)**: คอมไพเลอร์จะแจงส่วนไวยากรณ์ ตรวจสอบความถูกต้องทางภาษาศาสตร์ เพิ่มประสิทธิภาพของโค้ด และสร้างรหัสภาษาแอสเซมบลีเฉพาะของสถาปัตยกรรมฮาร์ดแวร์เป้าหมาย (`.s`)
3. **การแอสเซมเบิล (Assembly: `as`)**: ตัวแอสเซมเบลอร์จะแปลงคำสั่งแอสเซมบลีให้เป็นไฟล์ Object Code ระดับภาษาเครื่องที่สามารถย้ายตำแหน่งได้ (`.o` / `.obj`)
4. **การลิงก์ (Linking: `ld`)**: ตัวลิงเกอร์จะนำไฟล์อ็อบเจกต์ที่คอมไพล์แล้วมาผสานรวมเข้ากับรูทีนของไลบรารีมาตรฐาน (เช่น ฟังก์ชัน `printf` จาก `libc`) และกำหนดแอดเดรสหน่วยความจำภายนอกที่ชัดเจน จนได้ไฟล์ไบนารีที่รันได้โดยสมบูรณ์ (Executable Binary เช่น `a.out` หรือ `.exe`)

เทคนิคการใช้เครื่องมือ: การตรวจสอบขั้นตอนการคอมไพล์ใน GCC/Clang

เราสามารถตรวจสอบไฟล์ผลลัพธ์ในแต่ละขั้นตอนย่อยของการคอมไพล์ได้ผ่านแฟล็กคำสั่ง:

`gcc -E main.c -o main.i` (หยุดหลังสิ้นสุดขั้น Preprocessing)

`gcc -S main.i -o main.s` (หยุดหลังแปลงเป็น Assembly)

`gcc -c main.s -o main.o` (หยุดหลังแปลงเป็น Object Code)

`gcc main.o -o program` (ลิงก์เข้าสู่ไฟล์รันโปรแกรมสุดท้าย)

1.3 การแก้ปัญหาเชิงขั้นตอนวิธีและมาตรฐานผังงาน (Flowcharts)

วิศวกรรมซอฟต์แวร์ที่รัดกุมเริ่มต้นด้วยแบบจำลอง **Input-Process-Output (IPO)**:

- **Input (ข้อมูลนำเข้า)**: ระบุข้อมูลที่ต้องรับเข้ามาจากผู้ใช้หรือสภาพแวดล้อม พร้อมกำหนดชนิดข้อมูลที่เหมาะสม
- **Process (กระบวนการประมวลผล)**: กำหนดขั้นตอนวิธีทางคณิตศาสตร์และตรรกศาสตร์ที่จำเป็นในการหาคำตอบ
- **Output (ผลลัพธ์ที่ได้)**: กำหนดรูปแบบการแสดงผลลัพธ์ ความแม่นยำ และปลายทางของการนำเสนอข้อมูล

ผังงาน (Flowchart) เป็นแผนภาพกราฟิกมาตรฐานสำหรับอธิบายตรรกะของขั้นตอนวิธีก่อนลงมือเขียนโค้ด โดยมีสัญลักษณ์มาตรฐาน ISO/ANSI ดังนี้:

- **จุดเริ่มต้น / สิ้นสุด (Terminal: วงรีหรือสี่เหลี่ยมมุมมน)**: ระบุจุด `Start` หรือ `End` ของโปรแกรมหรือฟังก์ชัน
- **การรับข้อมูล / แสดงผล (I/O: สี่เหลี่ยมด้านขนาน)**: แสดงการอ่านค่าข้อมูลเข้า (เช่น `scanf`) หรือการแสดงผล (เช่น `printf`)
- **การประมวลผล (Process: สี่เหลี่ยมผืนผ้า)**: แสดงการคำนวณทางคณิตศาสตร์ การกำหนดค่าตัวแปร หรือการจัดการหน่วยความจำ
- **การตัดสินใจ (Decision: สี่เหลี่ยมขนมเปียกปูน)**: แสดงการแยกสาขาเงื่อนไข โดยมีเส้นทางออกกำกับด้วย `True/False` หรือ `Yes/No`
- **จุดเชื่อมต่อ (Connector: วงกลมขนาดเล็ก)**: ใช้เชื่อมโยงเส้นทางโฟลว์ที่มาบรรจบกันเพื่อหลีกเลี่ยงเส้นทับซ้อน
- **ทิศทางการทำงาน (Flowline: ลูกศร)**: แสดงลำดับทิศทางการประมวลผลของคำสั่ง

1.4 โครงสร้างทางกายวิภาคของโปรแกรมภาษา C & สตรีม I/O

โปรแกรมภาษา C มาตรฐานทุกโปรแกรมต้องมีฟังก์ชันจุดเริ่มต้นชื่อ `main` เสมอ โดยการจัดการ I/O จะเรียกใช้ฟังก์ชันจากเฮดเดอร์ `<stdio.h>`:

```c
#include <stdio.h>  /* ประกาศโพรโทไทป์ของ printf และ scanf */

int main(void) {
    double celsius, fahrenheit;

    printf("Enter temperature in Celsius: ");
    /* ข้อสังเกต: scanf จำเป็นต้องรับแอดเดรสหน่วยความจำ (&) ของตัวแปร */
    if (scanf("%lf", &celsius) == 1) {
        fahrenheit = (celsius * 9.0 / 5.0) + 32.0;
        printf("%.2f C = %.2f F\n", celsius, fahrenheit);
    } else {
        printf("Error: Invalid numerical input.\n");
        return 1; /* ค่าไม่เท่ากับศูนย์ระบุว่าโปรแกรมทำงานล้มเหลว */
    }

    return 0; /* ค่าศูนย์ระบุว่าโปรแกรมทำงานเสร็จสมบูรณ์เรียบร้อย */
}
```

ตัวอย่างผลลัพธ์การรันโปรแกรม (SAMPLE OUTPUT)

```
Enter temperature in Celsius: 28.5
28.50 C = 83.30 F
```

หัวใจสำคัญ: ตัวดำเนินการหาตำแหน่งแอดเดรส (&) ในคำสั่ง scanf

ในขณะที่ `printf` รับสำเนาค่าข้อมูลของตัวแปรไปแสดงผล แต่ `scanf` จำเป็นต้องเข้าไปเขียนแก้ไขค่าลงในหน่วยความจำของตัวแปรนั้นโดยตรง ดังนั้นการส่งชื่อตัวแปรโดยไม่มีเครื่องหมาย `&` (ยกเว้นชื่ออาร์เรย์และพอยน์เตอร์) จะเป็นการส่งค่าข้อมูลแทนแอดเดรส ซึ่งก่อให้เกิดข้อผิดพลาดร้ายแรงประเภท Undefined Behavior หรือเกิด Segmentation Fault จนโปรแกรมล่มทันที

บทที่ 2

ชนิดข้อมูล, ค่าลิเทอรัล, นิพจน์ & ตัวดำเนินการ

2.1 ตัวระบุ, คำสงวน และการแทนค่าข้อมูล

ตัวระบุ (Identifiers) คือชื่อที่ผู้เขียนโปรแกรมตั้งขึ้นสำหรับตัวแปร ฟังก์ชัน และชนิดข้อมูล โดยในภาษา C มีกฎเกณฑ์ดังนี้:

- ต้องขึ้นต้นด้วยตัวอักษรภาษาอังกฤษ (`a-z` , `A-Z`) หรือเครื่องหมายขีดล่าง (`_`)
- อักขระตัวถัดไปสามารถเป็นตัวอักษร ตัวเลข หรือเครื่องหมายขีดล่าง ห้ามมีช่องว่างหรือสัญลักษณ์พิเศษ
- คำนึงถึงตัวพิมพ์เล็ก-พิมพ์ใหญ่ (Case-Sensitive): `TotalScore` , `totalScore` , และ `TOTALSCORE` ถือเป็นคนละตัวแปรกัน
- ต้องไม่ซ้ำกับคำสงวน (Keywords) ทั้ง 32 คำของมาตรฐาน C89 (เช่น `auto` , `int` , `break` , `sizeof` , `typedef`)

2.2 ชนิดข้อมูลพื้นฐานและมาตรฐานขนาดคงที่ (Fixed-Width Standards)

มาตรฐานภาษา C ไม่ได้กำหนดขนาดไบต์ของชนิดข้อมูลพื้นฐานไว้อย่างตายตัว แต่กำหนดเพียงช่วงค่าขั้นต่ำไว้เท่านั้น ดังนั้นเพื่อความเข้ากันได้ข้ามระบบสถาปัตยกรรม มาตรฐาน C99 จึงได้นำเสนอเฮดเดอร์ `<stdint.h>` ซึ่งระบุชนิดข้อมูลจำนวนเต็มที่มีขนาดบิตคงที่แน่นอน:

ตัวระบุชนิดข้อมูล	ความหมายตามมาตรฐาน	ขนาดไบต์ทั่วไป (x86_64)	รหัสจัดรูปแบบ	ช่วงข้อมูลที่รองรับ
`char`	หน่วยข้อมูลที่เล็กที่สุด / อักขระ ASCII	1 ไบต์ (8 บิต)	`%c` / `%d`	-128 ถึง 127 (หรือ 0 ถึง 255)
`short int`	จำนวนเต็มขนาดสั้น	2 ไบต์ (16 บิต)	`%hd`	-32,768 ถึง 32,767
`int`	จำนวนเต็มมาตรฐานตามสถาปัตยกรรมเครื่อง	4 ไบต์ (32 บิต)	`%d` / `%i`	-2,147,483,648 ถึง 2,147,483,647
`long int`	จำนวนเต็มขนาดยาว	8 ไบต์ (64 บิต บน Linux)	`%ld`	$\approx -9.22 \times 10^{18}$ ถึง 9.22×10^{18}
`float`	จุดทศนิยมความละเอียดเดี่ยวตามมาตรฐาน IEEE 754	4 ไบต์ (32 บิต)	`%f`	ความแม่นยำประมาณ 6-7 ตำแหน่งทศนิยม
`double`	จุดทศนิยมความละเอียดคู่ตามมาตรฐาน IEEE 754	8 ไบต์ (64 บิต)	`%lf` (`%f` ใน printf)	ความแม่นยำประมาณ 15-17 ตำแหน่งทศนิยม
`int32_t`	จำนวนเต็มมีเครื่องหมายขนาดคงที่ 32 บิตแน่นอน	4 ไบต์ (32 บิต)	`PRId32`	-2,147,483,648 ถึง 2,147,483,647
`uint64_t`	จำนวนเต็มบวกไม่มีเครื่องหมายขนาดคงที่ 64 บิต	8 ไบต์ (64 บิต)	`PRIu64`	0 ถึง 18,446,744,073,709,551,615
`size_t`	จำนวนเต็มบวกที่คืนค่าจากตัวดำเนินการ `sizeof`	8 ไบต์ (บน 64 บิต)	`%zu`	รองรับขนาดความจุสูงสุดของวัตถุบนหน่วยความจำ

2.3 ตัวดำเนินการ, การตัดเศษการหาร & ตัวดำเนินการ Prefix เทียบกับ Postfix

ตัวดำเนินการทำหน้าที่ประมวลผลข้อมูลร่วมกับโอเปอแรนด์ โดยแบ่งตามจำนวนโอเปอแรนด์เป็น เอกภาค (Unary: 1 ตัว), ทวิภาค (Binary: 2 ตัว), และตรีภาค (Ternary: 3 ตัว)

การหารจำนวนเต็มเทียบกับการหารทศนิยม

เมื่อโอเปอแรนด์ทั้งสองฝั่งของตัวดำเนินการหาร (`/`) เป็นจำนวนเต็ม ภาษา C จะทำ **การหารจำนวนเต็มแบบตัดเศษทิ้ง (Truncating Integer Division)** โดยปัดเศษทศนิยมทิ้งเข้าหาศูนย์เสมอ (เช่น `7 / 2` จะได้ผลลัพธ์เป็น `3` ไม่ใช่ `3.5`) การหารจะได้ทศนิยมก็ต่อเมื่อมีโอเปอแรนด์อย่างน้อยหนึ่งตัวเป็นชนิดทศนิยม (เช่น `7.0 / 2` จะได้ `3.5`) ส่วนตัวดำเนินการมอดุโล (`%`) จะหาเศษที่เหลือจากการหารและใช้ได้เฉพาะกับจำนวนเต็มเท่านั้น

ตัวดำเนินการเพิ่มค่า Prefix (++x) เทียบกับ Postfix (x++)

ตัวดำเนินการเพิ่มค่า (`++`) และลดค่า (`--`) ทำการปรับค่าตัวแปรทีละ 1 โดยตำแหน่งของตัวดำเนินการมีผลต่อการประเมินค่านิพจน์:

- **Prefix (`++x`)**: ทำการเพิ่มค่าให้กับ `x` ก่อน แล้วจึงส่งค่านั้นไปประเมินในนิพจน์
- **Postfix (`x++`)**: ส่งค่า ปัจจุบัน ของ `x` ไปประเมินในนิพจน์ก่อน แล้วจึงทำการเพิ่มค่าให้กับ `x` ในลำดับถัดไป

```c
int a = 5, b = 5;
int res1 = ++a;  /* a กลายเป็น 6 ก่อน, จากนั้น res1 ได้รับค่า 6 */
int res2 = b++;  /* res2 ได้รับค่า 5 ก่อน, จากนั้น b จึงกลายเป็น 6 */
```

กับดักข้อผิดพลาด: พฤติกรรมที่ไม่พึงประสงค์ (UB) จากการแก้ไขตัวแปรซ้ำซ้อนในนิพจน์เดียว

นิพจน์ที่มีการแก้ไขค่าตัวแปรตัวเดิมมากกว่า 1 ครั้งระหว่าง Sequence Points เดียวกัน (เช่น `x = x++` หรือ `printf("%d %d", ++i, i++)`) ถูกจัดอยู่ในหมวด **Undefined Behavior (UB)** ตามมาตรฐาน ISO C คอมไพเลอร์มีสิทธิ์ตามกฎหมายที่จะสร้างโค้ดที่มีผลลัพธ์ผิดพลาด ขัดแย้ง หรือคาดเดาไม่ได้ จึงห้ามเขียนคำสั่งแก้ไขตัวแปรซ้ำในประโยคเดียวกันโดยเด็ดขาด!

2.4 การแปลงชนิดข้อมูลและการแคสต์ค่าอย่างชัดเจน (Explicit Casting)

ภาษา C รองรับทั้งการเลื่อนระดับชนิดข้อมูลอัตโนมัติ (Implicit Promotion) และการแปลงค่าอย่างเจาะจงโดยผู้เขียนโปรแกรม (Explicit Casting):

- **การเลื่อนระดับจำนวนเต็ม (Integer Promotion)**: ชนิดข้อมูลที่เล็กกว่า `int` (เช่น `char` , `short`) จะถูกขยายขนาดให้กลายเป็น `int` โดยอัตโนมัติขณะคำนวณ
- **การแปลงค่าตามลำดับเลขคณิต (Usual Arithmetic Conversions)**: ในการคำนวณที่ผสมชนิดข้อมูล โอเปอแรนด์ที่มีลำดับขั้นต่ำกว่าจะถูกแปลงให้เทียบเท่ากับชนิดข้อมูลที่สูงกว่าโดยอัตโนมัติ (เช่น `int + double` ค่า int จะถูกแปลงเป็น double)
- **การแคสต์ค่าอย่างชัดเจน (Explicit Casting)**: เขียนในรูป `(target_type)expression` สำคัญอย่างยิ่งในการป้องกันการตัดเศษจำนวนเต็ม:

```c
int sum = 25, count = 4;
/* หากไม่แคสต์: sum / count จะได้ 6.0 จากการหารตัดเศษจำนวนเต็ม */
double average = (double)sum / count; /* ได้ผลลัพธ์ที่ถูกต้องสมบูรณ์คือ 6.25 */
```

บทที่ 3

การตัดสินใจแยกสาขา, ตรรกศาสตร์ & การควบคุมเงื่อนไข

3.1 ความหมายเชิงตรรกะของบูลีนและตัวดำเนินการตรรกศาสตร์

ในมาตรฐานภาษา C89 ดั้งเดิมไม่มีชนิดข้อมูล `bool` โดยตรง แต่จะใช้หลักการตีความค่าตัวเลขทางตรรกะแทน:

- **เท็จ (False)**: แทนด้วยค่าจำนวนเต็ม `0` (หรือพอยน์เตอร์ `NULL`) เท่านั้น
- **จริง (True)**: แทนด้วย **ค่าตัวเลขอื่น ๆ ทั้งหมดที่ไม่ใช่ศูนย์** (โดยตัวดำเนินการเปรียบเทียบจะคืนค่า `1` เพื่อแทนความจริง)
- ตั้งแต่มาตรฐาน C99 เป็นต้นมา การนำเข้า `<stdbool.h>` จะมีชนิดข้อมูล `bool`, ค่า `true` (คือ `1`), และ `false` (คือ `0`) ให้ใช้งานอย่างเป็นทางการ

หมวดหมู่ตัวดำเนินการ	ตัวดำเนินการ	คำอธิบาย	ลำดับความสำคัญ
ความเท่ากัน (Equality)	`==`, `!=`	เท่ากับ, ไม่เท่ากับ	ต่ำ
การเปรียบเทียบ (Relational)	`<`, `<=`, `>`, `>=`	น้อยกว่า, น้อยกว่าหรือเท่ากับ, มากกว่า, มากกว่าหรือเท่ากับ	ปานกลาง-ต่ำ
ตรรกะ AND	`&&`	เป็นจริงก็ต่อเมื่อโอเปอแรนด์ทั้งสองฝั่งเป็นจริงทั้งคู่	ต่ำลงไปอีก
ตรรกะ OR	`||`	เป็นจริงเมื่อมีโอเปอแรนด์ฝั่งใดฝั่งหนึ่งเป็นจริง	ต่ำยิ่งกว่า
ตรรกะ NOT	`!`	กลับค่าความจริง (ตัวดำเนินการเอกภาค Prefix)	สูง

3.2 การประเมินผลลัพธ์แบบลัดวงจร (Short-Circuit Evaluation)

ตัวดำเนินการตรรกะ AND (`&&`) และ OR (`||`) รับประกันพฤติกรรมโครงสร้างแบบลัดวงจร:

- สำหรับ `expr1 && expr2`: หาก `expr1` มีค่าเป็นเท็จ (`0`) นิพจน์ทั้งหมดไม่มีทางเป็นจริงได้ ดังนั้น **ระบบจะไม่ประเมินค่า `expr2` อย่างเด็ดขาด**
- สำหรับ `expr1 || expr2`: หาก `expr1` มีค่าเป็นจริง (ไม่เท่ากับศูนย์) นิพจน์ทั้งหมดการันตีว่าเป็นจริงแล้ว ดังนั้น **ระบบจะข้ามการประเมินค่า `expr2` ไปทันที**

> **แบบแผนมืออาชีพ: การป้องกันการดีรีเฟอเรนซ์พอยน์เตอร์ด้วย Short-Circuiting**
> การประเมินแบบลัดวงจรช่วยให้เราสามารถเขียนคำสั่งตรวจสอบความปลอดภัยของพอยน์เตอร์ก่อนเข้าถึงข้อมูลได้อย่างมั่นใจ:
> `if (ptr != NULL && *ptr > 0) { /* การดีรีเฟอเรนซ์ตรงนี้ปลอดภัย 100% เพราะหาก ptr เป็น NULL จะไม่ถูกประเมิน */ }`

3.3 โครงสร้างแยกสาขา: บันได if-else เทียบกับตัวดำเนินการ Ternary

การทำงานแบบมีเงื่อนไขช่วยแยกทิศทางการประมวลผลของโปรแกรม:

```c
/* โครงสร้างบันไดเงื่อนไข if-else */
if (score >= 90) {
    grade = A;
} else if (score >= 80) {
    grade = B;
} else if (score >= 70) {
    grade = C;
} else {
    grade = F;
}

/* ตัวดำเนินการเงื่อนไขแบบ Ternary: (เงื่อนไข) ? ค่าเมื่อจริง : ค่าเมื่อเท็จ */
int max_val = (a > b) ? a : b;
```

3.4 โครงสร้าง switch-case & กลไกการไหลทะลุผ่าน (Fallthrough)

คำสั่ง `switch` รองรับการแยกสาขาหลายทางตามค่านิพจน์จำนวนเต็ม (`int` , `char` , หรือ `enum`) โดยห้ามใช้กับทศนิยมหรือสตริงเด็ดขาด:

```c
switch (command_code) {
    case W:
    case w:
        move_up();
        break; /* คำสั่ง break จำเป็นต้องมีเพื่อป้องกันการไหลทะลุไปยังเคสถัดไป */
    case S:
    case s:
        move_down();
        break;
    default:
        printf("Invalid motion command.\n");
        break;
}
```

ข้อควรระวัง: ภาวะเคสไหลทะลุโดยไม่ตั้งใจ (Accidental Fallthrough)

การละเว้นคำสั่ง `break` จะทำให้การประมวลผลไหลลงไปยังบล็อกคำสั่งของเคสถัดไปทันทีโดยไม่สนใจว่าเงื่อนไขจะตรงกันหรือไม่ หากตั้งใจให้ไหลทะลุผ่านเพื่อใช้โค้ดร่วมกัน ควรใส่คอมเมนต์กำกับอย่างชัดเจนเสมอ เช่น `/* FALLTHROUGH */` หรือใช้แอตทริบิวต์ `[[fallthrough]];` ของมาตรฐาน C23

บทที่ 4

การวนซ้ำ, ลูป & กลไกการทำซ้ำ

4.1 การจำแนกประเภทลูป: Pre-Test เทียบกับ Post-Test

โครงสร้างการวนซ้ำทำหน้าที่ประมวลผลชุดคำสั่งซ้ำตราบใดที่เงื่อนไขยังคงเป็นจริง โดยภาษา C มีลูป 3 รูปแบบหลัก:

- `while` **(ลูปตรวจสอบก่อนทำ: Pre-Test)**: ประเมินเงื่อนไขก่อนเข้าสู่ตัวลูป หากเริ่มต้นเป็นเท็จ ตัวลูปจะไม่ถูกประมวลผลเลยแม้แต่รอบเดียว
- `for` **(ลูปนับจำนวนรอบแบบ Pre-Test)**: รวมการกำหนดค่าเริ่มต้น เงื่อนไขตรวจสอบ และการก้าวเพิ่มค่าไว้ในส่วนหัวบรรทัดเดียวอย่างกระชับ
- `do-while` **(ลูปตรวจสอบหลังทำ: Post-Test)**: ประเมินเงื่อนไขที่ส่วนท้ายหลังตัวลูปทำงานเสร็จ **รับประกันว่าตัวลูปจะถูกประมวลผลอย่างน้อย 1 รอบเสมอ**

ประเภทลูป	ไวยากรณ์มาตรฐาน	จำนวนรอบขั้นต่ำ	กรณีการใช้งานหลัก
`while`	`while (condition) { ... }`	0	ลูปที่ขับเคลื่อนด้วยเหตุการณ์ภายนอก การอ่านสตรีมไฟล์จนกว่าจะพบ EOF
`for`	`for (init; cond; step) { ... }`	0	การวนรอบที่ทราบจำนวนรอบแน่นอน การท่องผ่านสมาชิกของอาร์เรย์ตามดัชนี
`do-while`	`do { ... } while (cond);`	1	เมนูอินเทอร์แอคทีฟ การตรวจสอบความถูกต้องของข้อมูลที่ต้องรับค่าก่อนอย่างน้อยหนึ่งครั้ง

4.2 การรบกวนลำดับการวนลูป: break เทียบกับ continue

ภาษา C มีคำสั่งกระโดด 2 คำสั่งสำหรับการปรับเปลี่ยนทิศทางในลูป:

- `break` : สั่งยุติการทำงานของลูปชั้นในสุดทันที และส่งการควบคุมไปยังคำสั่งถัดไปที่อยู่ต่อท้ายลูป
- `continue` : สั่งข้ามคำสั่งที่เหลือในรอบการทำงานปัจจุบัน และกระโดดไปยังส่วนการก้าวค่า (ใน `for`) หรือส่วนตรวจสอบเงื่อนไข (ใน `while` / `do-while`) เพื่อเริ่มรอบถัดไปทันที

```c
/* การหาจำนวนเฉพาะโดยใช้คำสั่งควบคุมลูป */
for (int n = 2; n <= 50; n++) {
    int is_prime = 1;
    for (int d = 2; d * d <= n; d++) {
        if (n % d == 0) {
            is_prime = 0;
            break; /* พบตัวหารลงตัว ยุติลูปในทันทีก่อนกำหนด */
        }
    }
    if (!is_prime) {
        continue; /* ข้ามการแสดงผลสำหรับจำนวนที่ไม่ใช่จำนวนเฉพาะ */
    }
    printf("%d ", n);
}
```

ผลลัพธ์การรันคำสั่ง (EXECUTION OUTPUT)

```
2 3 5 7 11 13 17 19 23 29 31 37 41 43 47
```

4.3 ลูปซ้อนลูป (Nested Loops) และตรรกะเชิงพิกัด 2 มิติ

เมื่อมีลูปหนึ่งตัวบรรจุอยู่ภายในตัวลูปอีกตัวหนึ่ง จะเรียกว่า **ลูปซ้อนลูป (Nested Loop)** โดยในการวนซ้ำของลูปนอกแต่ละรอบ ลูปในจะทำงานตั้งแต่ต้นจนครบรอบการทำงานทั้งหมดอย่างสมบูรณ์:

```c
/* การสร้างตารางสูตรคูณในรูปแบบพิกัดแถวและคอลัมน์ */
for (int row = 1; row <= 4; row++) {
    for (int col = 1; col <= 5; col++) {
        printf("%4d", row * col);
    }
    printf("\n"); /* ขึ้นบรรทัดใหม่เมื่อสิ้นสุดแต่ละแถว */
}
```

บทที่ 5

อาร์เรย์หนึ่งมิติ & ขั้นตอนวิธีพื้นฐาน

5.1 สถาปัตยกรรมหน่วยความจำแบบต่อเนื่องของอาร์เรย์

อาร์เรย์ (Array) คือกลุ่มของสมาชิกข้อมูลที่มีชนิดข้อมูลเดียวกัน มีขนาดคงที่แน่นอน และถูกจัดเก็บเรียงลำดับต่อเนื่องกันบนหน่วยความจำหลักโดยไม่มีช่องว่างคั่น ด้วยคุณลักษณะนี้ ทำให้ตำแหน่งแอดเดรสในหน่วยความจำของสมาชิกตัวที่ *i* สามารถคำนวณได้ทันทีในเวลาคงที่ *O(1)* ผ่านการคำนวณทางพอยน์เตอร์:

$$\text{Address}(\&arr[i]) = \text{Base_Address}(arr) + (i \times \text{sizeof}(\text{Element_Type}))$$

สมาชิก	ดัชนี (Index)	แอดเดรสสมมติบนหน่วยความจำ	ระยะออฟเซต (Byte Offset)	ค่าข้อมูลที่จัดเก็บ
`arr[0]`	0 (ฐาน Base)	`0x1000`	+0 ไบต์	`42`
`arr[1]`	1	`0x1004`	+4 ไบต์	`87`
`arr[2]`	2	`0x1008`	+8 ไบต์	`19`
`arr[3]`	3	`0x100C`	+12 ไบต์	`95`

5.2 การประกาศอาร์เรย์, การกำหนดค่าเริ่มต้น & การเติมค่าศูนย์ (Zero-Filling)

ไวยากรณ์การประกาศอาร์เรย์ประกอบด้วยชนิดข้อมูล ชื่ออาร์เรย์ และขนาดความจุภายในวงเล็บเหลี่ยม:

```
int a[5];                       /* ไม่ได้กำหนดค่าเริ่มต้น: มีค่าขยะ (Garbage) ที่คาดเดาไม่ได้ */
int b[5] = {10, 20, 30, 40, 50}; /* กำหนดค่าเริ่มต้นครบทุกช่องอย่างสมบูรณ์ */
int c[5] = {10, 20};             /* กำหนดบางส่วน: c[0]=10, c[1]=20 และ c[2..4] จะถูกเติมศูนย์อัตโนมัติ */
int d[5] = {0};                  /* สำนวนมาตรฐานสำหรับการเซ็ตสมาชิกทุกช่องให้เป็นศูนย์ 0 */
int e[]  = {1, 2, 3, 4};         /* คอมไพเลอร์คำนวณขนาดให้เอง: sizeof(e)/sizeof(e[0]) == 4 */
```

กับดักมรณะในภาษา C: การเข้าถึงข้อมูลเกินขอบเขตอาร์เรย์ (Out-of-Bounds Memory Corruption)

คอมไพเลอร์ภาษา C ไม่มีการสร้างโค้ดตรวจสอบขอบเขตอาร์เรย์ (Bounds-Checking) เลยแม้แต่น้อย การเข้าถึง `arr[5]` บนอาร์เรย์ขนาด 5 ช่อง จะส่งผลให้ระบบเข้าไปอ่านหรือเขียนทับหน่วยความจำของตัวแปรตัวอื่นบนสแตก ซึ่งก่อให้เกิดความเสียหายของข้อมูล การทำงานผิดเพี้ยน หรือช่องโหว่ความปลอดภัยระดับวิกฤต (Buffer Overflows)

5.3 ขั้นตอนวิธีพื้นฐานของอาร์เรย์: การค้นหา, ค่าน้อยสุด/มากสุด & การกลับด้านในตัว

โปรแกรมต่อไปนี้สาธิต 4 ขั้นตอนวิธีหลัก: การคำนวณผลรวม, ค่าเฉลี่ย, การหาค่าสุดขีด และการกลับด้านข้อมูลในตัวด้วยเทคนิค Two-Pointer:

```c
#include <stdio.h>

int main(void) {
    int nums[] = {45, 12, 89, 34, 78, 23, 99, 56};
    int n = sizeof(nums) / sizeof(nums[0]);

    /* 1. คำนวณผลรวม, ค่าน้อยสุด, และค่ามากสุด */
    int sum = 0, min_val = nums[0], max_val = nums[0];
    for (int i = 0; i < n; i++) {
        sum += nums[i];
        if (nums[i] < min_val) min_val = nums[i];
        if (nums[i] > max_val) max_val = nums[i];
    }
    printf("Count: %d | Sum: %d | Avg: %.2f | Min: %d | Max: %d\n",
           n, sum, (double)sum / n, min_val, max_val);

    /* 2. การกลับด้านข้อมูลในตัว (In-Place Reversal) ด้วยเทคนิคสลับสองพอยน์เตอร์ */
    for (int left = 0, right = n - 1; left < right; left++, right--) {
        int temp = nums[left];
        nums[left] = nums[right];
        nums[right] = temp;
    }

    printf("Reversed: ");
    for (int i = 0; i < n; i++) printf("%d ", nums[i]);
    printf("\n");
    return 0;
}
```

ผลลัพธ์การรันคำสั่ง (EXECUTION OUTPUT)

```
Count: 8 | Sum: 436 | Avg: 54.50 | Min: 12 | Max: 99
Reversed: 56 99 23 78 34 89 12 45
```

บทที่ 6

อาร์เรย์หลายมิติ & การคำนวณเมทริกซ์

6.1 เมทริกซ์ 2 มิติและโครงสร้างการจัดเก็บบนหน่วยความจำแบบ Row-Major

อาร์เรย์สองมิติแทนโครงสร้างตารางของแถวและคอลัมน์ (`type arr[ROWS][COLS]`) แม้ในเชิงมโนทัศน์เราจะมองเป็นตารางสี่เหลี่ยม แต่ในหน่วยความจำ RAM ทางกายภาพนั้นเป็นพื้นที่แอดเดรสแบบมิติเดียวเส้นตรง ภาษา C จึงจัดเก็บอาร์เรย์หลายมิติตามลำดับ **Row-Major Order**: นั่นคือสมาชิกทุกตัวของแถวที่ 0 จะเรียงติดต่อกันจนครบ จากนั้นจึงตามด้วยสมาชิกทั้งหมดของแถวที่ 1 ต่อเนื่องกันไปเรื่อย ๆ

$$\texttt{Flat_Offset(arr[r][c]) = (r × COLS) + c}$$

6.2 การคูณเมทริกซ์ ($A_{m\times k} \times B_{k\times n} = C_{m\times n}$)

การคูณเมทริกซ์ใช้ลูปซ้อนกัน 3 ชั้น: ลูปนอกสุดท่องตามแถวของเมทริกซ์ A, ลูปกลางท่องตามคอลัมน์ของเมทริกซ์ B, และลูปในสุดทำหน้าที่สะสมผลคูณจุด (Dot Product) ตามมิติร่วม k:

```c
#include <stdio.h>
#define R1 2
#define C1_R2 3
#define C2 2

int main(void) {
    int A[R1][C1_R2] = { {1, 2, 3}, {4, 5, 6} };
    int B[C1_R2][C2] = { {7, 8}, {9, 1}, {2, 3} };
    int C[R1][C2] = {{0}};

    for (int i = 0; i < R1; i++) {
        for (int j = 0; j < C2; j++) {
            for (int k = 0; k < C1_R2; k++) {
                C[i][j] += A[i][k] * B[k][j];
            }
        }
    }

    printf("Product Matrix C (%dx%d):\n", R1, C2);
    for (int i = 0; i < R1; i++) {
        for (int j = 0; j < C2; j++) {
            printf("%5d", C[i][j]);
        }
        printf("\n");
    }
    return 0;
}
```

ผลลัพธ์การรันคำสั่ง (EXECUTION OUTPUT)

```
Product Matrix C (2x2):
31 19
85 55
```

เทคนิคด้านประสิทธิภาพ: ลำดับลูปที่เป็นมิตรต่อแคช (Cache-Friendly Loop Order)

เนื่องจากหน่วยความจำถูกจัดเก็บแบบ Row-Major การเข้าถึงข้อมูลเรียงตามแถว (โดยให้ดัชนีคอลัมน์ `j` เปลี่ยนแปลงในลูปชั้นในสุด `C[i][j]`) จะทำให้เกิด CPU Cache Hits สูงสุด การสลับลำดับลูปให้เข้าถึงตามคอลัมน์ก่อนจะก่อให้เกิด Cache Misses อย่างรุนแรง และทำให้ประสิทธิภาพลดลงอย่างมาก

บทที่ 7

สตริง, การประมวลผลข้อความ & ไลบรารี <string.h>

7.1 มโนทัศน์ของอักขระศูนย์ (Null Terminator) และลิเทอรัลสตริง

ในภาษา C ไม่มีชนิดข้อมูลสตริงระดับอ็อบเจกต์ แต่ **สตริง** ถูกนิยามตามแบบแผนว่าเป็นอาร์เรย์ของ `char` หนึ่งมิติที่ปิดท้ายด้วย **อักขระว่าง (Null Character: `'\0'` รหัส ASCII คือ 0)** เสมอ โดยอักขระ Null Terminator นี้จะเป็นสัญญาณบอกให้ฟังก์ชันในไลบรารีมาตรฐานทราบว่าข้อความสิ้นสุดลงตรงจุดใดในหน่วยความจำ

รูปแบบการประกาศ	การแทนค่า	ขนาดหน่วยความจำ	เซกเมนต์หน่วยความจำ & ความเปลี่ยนแปลงได้
`char c = A;`	ลิเทอรัลอักขระเดี่ยว (Single Quotes)	1 ไบต์	Stack; สามารถแก้ไขค่าได้
`char s[] = "A";`	อาร์เรย์ที่กำหนดค่าเริ่มต้นด้วยสตริง	2 ไบต์ (`A` และ `'\0'`)	Stack; สำเนาที่สามารถแก้ไขค่าได้
`const char *p = "A";`	พอยน์เตอร์ชี้ไปยังลิเทอรัลสตริง	พอยน์เตอร์ (8B) + ลิเทอรัล 2B	Read-Only Data Segment (`.rodata`); ห้ามแก้ไขค่าเด็ดขาด

7.2 การรับข้อมูลสตริงอย่างปลอดภัย: เลิกใช้ gets() และ scanf("%s")

ฟังก์ชันในอดีตอย่าง `gets()` ถูกถอดออกจากมาตรฐาน ISO C11 โดยเด็ดขาด เนื่องจากไม่สามารถจำกัดขนาดบัฟเฟอร์ได้ นำไปสู่ปัญหาบัฟเฟอร์โอเวอร์โฟลว์อย่างหลีกเลี่ยงไม่ได้ เช่นเดียวกับ `scanf("%s", buffer)` ที่หยุดอ่านเมื่อเจอช่องว่างและเสี่ยงต่อการเขียนเกินความจุของอาร์เรย์

มาตรฐานความปลอดภัยระดับอุตสาหกรรมสำหรับการรับข้อความสตริงคือการใช้ **`fgets()`** :

```c
char buffer[64];
printf("Enter full name: ");
if (fgets(buffer, sizeof(buffer), stdin) != NULL) {
    /* ตัดอักขระขึ้นบรรทัดใหม่ (
) ที่ fgets บันทึกติดมาออก */
    buffer[strcspn(buffer, "\n")] = '\0';
    printf("Hello, %s!\n", buffer);
}
```

7.3 ชุดฟังก์ชันมาตรฐานใน <string.h>

ไลบรารีมาตรฐาน `<string.h>` รวบรวมฟังก์ชันสำหรับจัดการข้อความที่สำคัญ:

- `size_t strlen(const char *s)` : คืนค่าจำนวนตัวอักษรโดยไม่นับรวม `'\0'` ที่ปิดท้าย
- `char *strcpy(char *dest, const char *src)` : คัดลอกข้อความจาก `src` ไปยัง `dest` (คำเตือน: ไม่ตรวจขนาดบัฟเฟอร์)
- `char *strncpy(char *dest, const char *src, size_t n)` : คัดลอกแบบจำกัดความยาว ป้องกันโอเวอร์โฟลว์ แต่ไม่รับประกันว่าจะใส่ `'\0'` หาก `strlen(src) >= n`
- `char *strcat(char *dest, const char *src)` : นำข้อความจาก `src` ไปต่อท้าย `dest`
- `int strcmp(const char *s1, const char *s2)` : เปรียบเทียบตามลำดับพจนานุกรม คืนค่า `0` หากเหมือนกันทุกประการ คืนค่าติดลบหาก *s1 < s2* และคืนค่าบวกหาก *s1 > s2*

```c
#include <stdio.h>
#include <string.h>
#include <ctype.h>

int main(void) {
    char text[100] = "Computer Systems C-101";
    int vowels = 0, consonants = 0, digits = 0;

    for (int i = 0; text[i] != '\0'; i++) {
        char ch = tolower(text[i]);
        if (isdigit(ch)) {
            digits++;
        } else if (isalpha(ch)) {
            if (ch == a || ch == e || ch == i || ch == o || ch == u)
                vowels++;
            else
                consonants++;
        }
    }
    printf("String: \"%s\"\n", text);
    printf("Vowels: %d | Consonants: %d | Digits: %d\n", vowels, consonants, digits);
    return 0;
}
```

ผลลัพธ์การรันคำสั่ง (EXECUTION OUTPUT)

```
String: "Computer Systems C-101"
Vowels: 5 | Consonants: 10 | Digits: 3
```

บทที่ 8

พอยน์เตอร์, โมเดลหน่วยความจำ & พอยน์เตอร์อาริทเมติก

8.1 พื้นที่แอดเดรสของโปรเซสและกลไกหน่วยความจำ

การจะเชี่ยวชาญภาษา C จำเป็นต้องเข้าใจการจัดสรรหน่วยความจำของโปรเซสที่คอมไพล์แล้ว โดยระบบปฏิบัติการจะจัดสรรพื้นที่แอดเดรสเสมือน (Virtual Memory) ให้แก่แต่ละโปรเซส ซึ่งแบ่งออกเป็นเซกเมนต์เชิงตรรกะที่ชัดเจน:

- **เซกเมนต์โค้ด (Text / Code Segment)**: พื้นที่หน่วยความจำแบบอ่านอย่างเดียว (Read-Only) ที่เก็บชุดคำสั่งไบนารีสำหรับ CPU
- **เซกเมนต์ Data & BSS**: เก็บตัวแปรโกลบอลและตัวแปร `static` โดยตัวแปรที่มีการกำหนดค่าเริ่มต้นจะอยู่ใน Data ส่วนตัวแปรที่ยังไม่ได้กำหนดค่าเริ่มต้นจะอยู่ใน BSS (ซึ่งระบบปฏิบัติการจะเติมค่า 0 ให้ทันทีเมื่อโปรแกรมเริ่มทำงาน)
- **หน่วยความจำฮีป (Heap)**: หน่วยความจำพลวัตที่จัดสรรขณะรันไทม์ผ่านฟังก์ชัน `malloc` และ `free` (ขยายตัวจากล่างขึ้นบนสู่แอดเดรสที่สูงขึ้น)
- **หน่วยความจำสแตก (Stack)**: เก็บตัวแปรโลคอลอัตโนมัติ, Stack Frames ของฟังก์ชัน และแอดเดรสสำหรับคืนค่าการทำงาน (ขยายตัวจากบนลงล่างสู่แอดเดรสที่ต่ำลง)

8.2 กายวิภาคของพอยน์เตอร์: ตัวดำเนินการหาตำแหน่ง (&) และการดีรีเฟอเรนซ์ (*)

พอยน์เตอร์ (Pointer) คือตัวแปรพิเศษที่ค่าของมันคือแอดเดรสตำแหน่งบนหน่วยความจำของตัวแปรตัวอื่น:

- **ตัวดำเนินการระบุแอดเดรส (Address-of Operator: `&`)**: ตัวดำเนินการเอกภาคที่ทำหน้าที่ดึงแอดเดรสหน่วยความจำ (ฐานสิบหก) ของตัวแปรเป้าหมาย
- **ตัวดำเนินการดีรีเฟอเรนซ์ (Dereference / Indirection Operator: `*`)**: ตัวดำเนินการเอกภาคที่ใช้เข้าถึงหรือแก้ไขค่าข้อมูลที่จัดเก็บอยู่ ณ ตำแหน่งแอดเดรสที่พอยน์เตอร์ชี้ไป

```c
int x = 42;
int *p = &x;      /* p ชี้ไปยังแอดเดรสของตัวแปร x */

printf("Value of x:       %d\n", x);     /* พิมพ์ค่า 42 */
printf("Address of x (&x): %p\n", (void*)&x);
printf("Value of p:       %p\n", (void*)p);  /* มีค่าเหมือน &x ทุกประการ */
printf("Value at *p:      %d\n", *p);    /* การดีรีเฟอเรนซ์ได้ค่า 42 */

*p = 100;         /* แก้ไขค่าของ x ทางอ้อมผ่านพอยน์เตอร์หน่วยความจำ */
printf("New value of x:   %d\n", x);     /* ขณะนี้ x มีค่าเป็น 100 */
```

> **กฎความปลอดภัยวิกฤต: NULL Pointers เทียบกับ Wild Pointers**
>
> พอยน์เตอร์ที่ประกาศโดยไม่ได้กำหนดค่าเริ่มต้น (เช่น `int *p;`) จะบรรจุบิตขยะที่คาดเดาไม่ได้และเรียกว่า **Wild Pointer** การดีรีเฟอเรนซ์ไวลด์พอยน์เตอร์จะไปทำลายหน่วยความจำส่วนอื่นของระบบโดยไม่รู้ตัว ดังนั้นควรตั้งค่าเริ่มต้นให้ชี้ไปยังแอดเดรสที่ถูกต้องหรือกำหนดเป็น `NULL` ทันทีเสมอ และต้องตรวจสอบพอยน์เตอร์ก่อนดีรีเฟอเรนซ์ทุกครั้ง: `if (p != NULL) { *p = 5; }`

8.3 พอยน์เตอร์อาริทเมติก & ทวิภาวะของอาร์เรย์และพอยน์เตอร์ (Array-Pointer Duality)

การคำนวณทางคณิตศาสตร์บนพอยน์เตอร์มีความแตกต่างอย่างสิ้นเชิงจากการคำนวณจำนวนเต็มทั่วไป เมื่อเรานำจำนวนเต็ม k ไปบวกกับพอยน์เตอร์ `p` ที่มีชนิดข้อมูลเป็น `T*` แอดเดรสทางกายภาพจะก้าวเลื่อนไปข้างหน้าเป็นจำนวน `k × sizeof(T)` ไบต์:

$$\texttt{p + k} \equiv \texttt{(uintptr_t)p + (k} \times \texttt{sizeof(*p))}$$

กฎคณิตศาสตร์นี้ก่อให้เกิด **ทวิภาวะของอาร์เรย์และพอยน์เตอร์ (Array-Pointer Duality)** ในภาษา C:

- ชื่ออาร์เรย์ `arr` จะลดรูป (Decay) กลายเป็นพอยน์เตอร์ที่ชี้ไปยังสมาชิกตัวแรกโดยปริยาย: `arr ≡ &arr[0]`
- การเข้าถึงข้อมูลด้วยวงเล็บดัชนีเป็นเพียงน้ำตาลทางไวยากรณ์ (Syntactic Sugar) ของพอยน์เตอร์อาริทเมติก: `arr[i] ≡ *(arr + i)`

```c
#include <stdio.h>

int main(void) {
    int data[4] = {10, 20, 30, 40};
    int *ptr = data; /* เทียบเท่ากับคำสั่ง: ptr = &data[0]; */

    for (int i = 0; i < 4; i++) {
        printf("data[%d] = %d | *(ptr + %d) = %d | Address: %p\n",
               i, data[i], i, *(ptr + i), (void*)(ptr + i));
    }
    return 0;
}
```

บทที่ 9

การเขียนโปรแกรมแบบโมดูลาร์, สโคป & ฟังก์ชัน

9.1 สถาปัตยกรรมแบบโมดูลาร์: โพรโทไทป์, นิยาม และการเรียกใช้งาน

การเขียนโปรแกรมแบบโมดูลาร์ช่วยแบ่งแยกโค้ดเบสขนาดใหญ่ให้กลายเป็นกระบวนการย่อยที่ทดสอบและบำรุงรักษาได้ง่าย เรียกว่า **ฟังก์ชัน (Functions)** โดยการใช้งานฟังก์ชันประกอบด้วย 3 ส่วนสำคัญ:

1. **การประกาศโพรโทไทป์ (Function Prototype)**: แจ้งให้คอมไพเลอร์ทราบถึงรูปแบบของฟังก์ชัน (ชนิดค่าคืนกลับ, ชื่อ, และชนิดพารามิเตอร์) ก่อนที่จะมีการเรียกใช้งาน
2. **นิยามของฟังก์ชัน (Function Definition)**: บล็อกคำสั่งการทำงานที่แท้จริงของฟังก์ชัน
3. **การเรียกใช้งานฟังก์ชัน (Function Call)**: จุดในโปรแกรมที่มีการส่งอาร์กิวเมนต์จริงเข้าไปประมวลผล

```c
/* 1. การประกาศโพรโทไทป์ */
double calculate_circle_area(double radius);

int main(void) {
    /* 3. การเรียกใช้งานฟังก์ชัน */
    double a = calculate_circle_area(5.0);
    printf("Area: %.2f\n", a);
    return 0;
}

/* 2. นิยามบล็อกการทำงานจริงของฟังก์ชัน */
double calculate_circle_area(double radius) {
    const double PI = 3.141592653589793;
    return PI * radius * radius;
}
```

9.2 ความหมายของการส่งผ่านค่า (Pass-by-Value) และความคงทนของตัวแปรต้นทาง

ภาษา C ใช้กลไก **Pass-by-Value** สำหรับพารามิเตอร์ทุกตัวอย่างเคร่งครัด เมื่อมีการส่งอาร์กิวเมนต์เข้าสู่ฟังก์ชัน ระบบจะสร้างสำเนาข้อมูลชุดใหม่ขึ้นมาอย่างอิสระบน Stack Frame ของฟังก์ชันปลายทาง การแก้ไขค่าตัวแปรพารามิเตอร์ภายในฟังก์ชันจะส่งผลกระทบเฉพาะสำเนาในฟังก์ชันเท่านั้น ตัวแปรดั้งเดิมของผู้เรียกจะไม่ได้รับผลกระทบใด ๆ ทั้งสิ้น

9.3 อายุการใช้งานตัวแปร, ขอบเขตสโคป และระยะเวลาแบบ Static

การเข้าถึงและการคงอยู่ของตัวแปรถูกควบคุมด้วยคลาสการจัดเก็บ (Storage Classes) และขอบเขตสโคป:

คลาสการจัดเก็บ	คีย์เวิร์ด	ขอบเขตการมองเห็น (Scope)	ช่วงอายุการคงอยู่ (Lifetime)	ค่าเริ่มต้นปริยาย
Automatic (ตัวแปรอัตโนมัติ)	`auto` (ปริยาย)	ภายในบล็อก / ฟังก์ชัน	ตั้งแต่สร้าง Stack Frame จนกระทั่งรีเทิร์น	ค่าขยะ (Garbage)
Static Local (สถิตเฉพาะที่)	`static`	ภายในบล็อกที่ประกาศ	คงอยู่ตลอดระยะเวลาที่โปรแกรมทำงาน	ศูนย์ (`0`)
Global / External (โกลบอล)	`extern`	มองเห็นได้จากทุกไฟล์ในแอปพลิเคชัน	คงอยู่ตลอดระยะเวลาที่โปรแกรมทำงาน	ศูนย์ (`0`)
File-Static (สถิตระดับไฟล์)	`static`	จำกัดการเข้าถึงเฉพาะภายในไฟล์ `.c` นั้น	คงอยู่ตลอดระยะเวลาที่โปรแกรมทำงาน	ศูนย์ (`0`)

```c
void counter_demo(void) {
    int auto_var = 0;
    static int static_var = 0; /* รักษาค่าเดิมไว้ได้แม้ฟังก์ชันจะทำงานจบไปแล้ว! */
    auto_var++;
    static_var++;
    printf("auto: %d | static: %d\n", auto_var, static_var);
}
```

บทที่ 10

จำลองการส่งผ่านแอดเดรส, อาร์เรย์ในฟังก์ชัน & ฟังก์ชันพอยน์เตอร์

10.1 การจำลอง Pass-by-Reference ผ่านการส่งแอดเดรสพอยน์เตอร์

เนื่องจากภาษา C ไม่มีชนิดข้อมูล Reference แท้แบบภาษา C++ (`&`) นักพัฒนาจึงจำลอง **Pass-by-Reference** ด้วยการส่งผ่านแอดเดรสหน่วยความจำอย่างชัดเจน:

1. ผู้เรียกส่งแอดเดรสของตัวแปรเข้าไปโดยใช้ตัวดำเนินการ Address-of (`&x`)
2. ฟังก์ชันปลายทางรับค่านั้นเข้ามาในฐานะพอยน์เตอร์พารามิเตอร์ (`int *p`)
3. ฟังก์ชันปลายทางแก้ไขข้อมูลในหน่วยความจำของผู้เรียกโดยตรงผ่านการดีรีเฟอเรนซ์ (`*p = val`)

```c
#include <stdio.h>

/* ฟังก์ชันสลับค่าคลาสสิกด้วยการส่งผ่านพอยน์เตอร์ */
void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

/* การคืนค่าผลลัพธ์หลายค่าผ่านพอยน์เตอร์พารามิเตอร์ */
void get_min_max(const int arr[], int size, int *out_min, int *out_max) {
    *out_min = arr[0];
    *out_max = arr[0];
    for (int i = 1; i < size; i++) {
        if (arr[i] < *out_min) *out_min = arr[i];
        if (arr[i] > *out_max) *out_max = arr[i];
    }
}

int main(void) {
    int x = 10, y = 99;
    swap(&x, &y);
    printf("After swap: x = %d, y = %d\n", x, y); /* x=99, y=10 */
    return 0;
}
```

10.2 การส่งอาร์เรย์เข้าฟังก์ชันและปรากฏการณ์ Array Decay

เมื่อมีการส่งอาร์เรย์เป็นอาร์กิวเมนต์เข้าสู่ฟังก์ชัน ตัวอาร์เรย์จะ **ลดรูป (Decay) กลายเป็นพอยน์เตอร์** ชี้ไปยังสมาชิกตัวแรกโดยอัตโนมัติ ดังนั้นภายในฟังก์ชัน คำสั่ง `sizeof(arr)` จะได้ขนาดของพอยน์เตอร์ (8 ไบต์บนระบบ 64 บิต) และสูญเสียข้อมูลความยาวที่แท้จริงของอาร์เรย์ไปทั้งหมด

> **กฎเหล็กภาคปฏิบัติ: ต้องส่งขนาดความยาวของอาร์เรย์แยกต่างหากเสมอ**
> เนื่องจากอาร์เรย์จะลดรูปกลายเป็นพอยน์เตอร์ทันทีเมื่อก้าวข้ามเข้าสู่ฟังก์ชัน คุณจึงต้องส่งขนาดความยาวของอาร์เรย์เข้ามาเป็นพารามิเตอร์ตัวที่สองเสมอ: `void process(int arr[], int size)`

10.3 ฟังก์ชันพอยน์เตอร์, คอลแบ็ก & ตารางดิสแพตช์ (Function Pointers)

ในภาษา C ฟังก์ชันถูกเก็บอยู่ในเซกเมนต์ Text และมีแอดเดรสบนหน่วยความจำที่แน่นอน **ฟังก์ชันพอยน์เตอร์ (Function Pointer)** คือตัวแปรที่สามารถเก็บแอดเดรสของฟังก์ชันได้ ทำให้เกิดความสามารถด้าน Runtime Polymorphism, Callback Architecture และ Dispatch Tables:

การประกาศไวยากรณ์และสถาปัตยกรรมคอลแบ็ก

```c
#include <stdio.h>
#include <stdlib.h>

/* ชนิดข้อมูลฟังก์ชันพอยน์เตอร์: ชี้ไปยังฟังก์ชันใดก็ได้ที่รับ int 2 ตัวและคืนค่า int */
typedef int (*BinaryOp)(int, int);

int add(int a, int b) { return a + b; }
int multiply(int a, int b) { return a * b; }

/* ฟังก์ชันอันดับสูงที่รับฟังก์ชันพอยน์เตอร์เข้ามาเป็นคอลแบ็ก */
int compute(int x, int y, BinaryOp operation) {
    return operation(x, y); /* เรียกทำงานคอลแบ็ก */
}

/* ฟังก์ชันเปรียบเทียบคอลแบ็กสำหรับ qsort ของ stdlib */
int ascending_compare(const void *a, const void *b) {
    int val_a = *(const int*)a;
    int val_b = *(const int*)b;
    return (val_a > val_b) - (val_a < val_b); /* ปลอดภัยจากการโอเวอร์โฟลว์ */
}

int main(void) {
    printf("Add: %d | Multiply: %d\n", compute(5, 4, add), compute(5, 4, multiply));

    int data[] = {50, 20, 40, 10, 30};
    qsort(data, 5, sizeof(int), ascending_compare);

    printf("Sorted via qsort callback: ");
    for (int i = 0; i < 5; i++) printf("%d ", data[i]);
    printf("\n");
    return 0;
}
```

ผลลัพธ์การรันคำสั่ง (EXECUTION OUTPUT)

```
Add: 9 | Multiply: 20
Sorted via qsort callback: 10 20 30 40 50
```

บทที่ 11

อีนัม, Typedef, โครงสร้างข้อมูลสตรักต์ & การจัดเรียง Alignment

11.1 การแจงนับ (enum) และการตั้งชื่อชนิดข้อมูลจำลอง (typedef)

การแจงนับ (Enumeration: `enum`) กำหนดชนิดข้อมูลจำนวนเต็มที่จำกัดเซตของค่าที่ถูกต้องไว้เฉพาะชุดค่าคงที่ที่มนุษย์อ่านเข้าใจได้ โดยปริยายแล้ว คอมไพเลอร์จะกำหนดค่าจำนวนเต็มเริ่มต้นที่ `0` และเพิ่มขึ้นทีละ 1 ต่อเนื่องกันไป:

```
enum Status { STATUS_OK = 0, STATUS_PENDING, STATUS_ERROR = 500, STATUS_TIMEOUT };
enum Status s = STATUS_PENDING; /* มีค่าเท่ากับ 1 */
```

คำสั่ง `typedef` ใช้สำหรับตั้งชื่อนามแฝง (Alias) ให้กับชนิดข้อมูลที่มีอยู่เดิม เพื่อลดความซับซ้อนของไวยากรณ์และเพิ่มความกระจ่างในการอ่านโค้ด:

```
typedef unsigned long long uint64;
typedef enum { DIR_NORTH, DIR_SOUTH, DIR_EAST, DIR_WEST } Direction;
```

11.2 โครงสร้างข้อมูลสตรักต์ (struct): การรวบรวมเรกคอร์ดข้อมูลต่างชนิด

ต่างจากอาร์เรย์ (ซึ่งบังคับให้สมาชิกทุกตัวต้องมีชนิดข้อมูลเดียวกัน) **สตรักต์ (Structure)** ทำหน้าที่ผูกโยงข้อมูลที่มีชนิดข้อมูลหลากหลายเข้าไว้ด้วยกันภายใต้การจัดวางหน่วยความจำที่เป็นผืนเดียว:

```
typedef struct {
    int day;
    int month;
    int year;
} Date;

typedef struct {
    int id;
    char name[50];
    float gpa;
    Date enrollment_date; /* การประกอบสตรักต์แบบซ้อนกัน (Nested Composition) */
} Student;
```

11.3 การเข้าถึงสมาชิก: ตัวดำเนินการจุด (.) เทียบกับตัวดำเนินการลูกศร (->)

การเข้าถึงสมาชิกของตัวแปรสตรักต์โดยตรงจะใช้ **ตัวดำเนินการจุด (Dot Operator: `.`)** แต่หากเข้าถึงสมาชิกผ่านพอยน์เตอร์ที่ชี้ไปยังสตรักต์ ภาษา C ได้จัดเตรียม **ตัวดำเนินการลูกศร (Arrow Operator: `->`)** เพื่อเป็นน้ำตาลทางไวยากรณ์แทนการดีรีเฟอเรนซ์ตามด้วยเครื่องหมายจุด:

$$\texttt{ptr->member} \equiv \texttt{(*ptr).member}$$

> **แนวทางปฏิบัติเพื่อประสิทธิภาพ: การส่งผ่านสตรักต์ด้วย const Pointer**
> การส่งตัวแปรสตรักต์ขนาดใหญ่เข้าฟังก์ชันแบบ Pass-by-Value จะทำการคัดลอกข้อมูลทุกไบต์ของทุกสมาชิกลงบนสแตก ซึ่งทำให้สิ้นเปลืองเวลาและหน่วยความจำอย่างมาก ควรสลับมาส่งผ่านด้วยพอยน์เตอร์แทนเสมอ และกำกับด้วย `const` หากฟังก์ชันนั้นต้องการเพียงแค่อ่านข้อมูล: `void print_student(const Student *s)`

11.4 การจัดเรียงหน่วยความจำ, ไบต์แพดดิง (Padding) และบิตฟิลด์ (Bit-Fields)

สถาปัตยกรรมฮาร์ดแวร์ซีพียูสามารถเข้าถึงหน่วยความจำได้อย่างรวดเร็วเมื่อข้อมูลถูกจัดวางตรงตามพหุคูณของขนาดข้อมูลนั้น ๆ (เช่น `int` ขนาด 4 ไบต์ จะต้องเริ่มที่แอดเดรสที่หารด้วย 4 ลงตัว) เพื่อให้เป็นไปตามข้อกำหนดการจัดเรียงนี้ คอมไพเลอร์จะแทรก **ไบต์เสริม (Padding Bytes)** เข้าไประหว่างสมาชิกของสตรักต์โดยอัตโนมัติ:

```c
struct Unpadded {
    char a;     /* 1 ไบต์ + คอมไพเลอร์แทรก padding 3 ไบต์! */
    int b;      /* 4 ไบต์ */
    char c;     /* 1 ไบต์ + padding ปิดท้ายอีก 3 ไบต์! */
}; /* ขนาดรวม: 12 ไบต์ แม้ข้อมูลจริงของผู้ใช้จะมีเพียง 6 ไบต์! */

struct Optimized {
    int b;      /* 4 ไบต์ */
    char a;     /* 1 ไบต์ */
    char c;     /* 1 ไบต์ + padding เพียง 2 ไบต์ */
}; /* ขนาดรวม: 8 ไบต์ (จัดเรียงสมาชิกลดหลั่นตามขนาด) */
```

บิตฟิลด์ (Bit-Fields) สำหรับฮาร์ดแวร์และการแพ็กข้อมูลแฟล็ก

บิตฟิลด์ช่วยให้สามารถบีบอัดแฟล็กหรือตัวเลขจำนวนเต็มหลาย ๆ ตัวให้ใช้พื้นที่บิตตามที่กำหนดจริงภายใน Word เดียว:

```c
struct DeviceStatus {
    unsigned int is_active    : 1;  /* 1 บิต: เก็บค่าได้ 0 หรือ 1 */
    unsigned int error_code   : 4;  /* 4 บิต: เก็บค่า 0 ถึง 15 */
    unsigned int channel_num  : 3;  /* 3 บิต: เก็บค่า 0 ถึง 7 */
}; /* บรรจุพอดีภายใน 32 บิต Word เดียวอย่างประหยัดหน่วยความจำ! */
```

บทที่ 12

ยูเนียน, การจัดการระดับบิต (Bitwise) & บิตฟิลด์

12.1 พฤติกรรมการแชร์หน่วยความจำของยูเนียน (union)

ยูเนียน (Union) มีไวยากรณ์ละม้ายคล้ายกับสตรักต์ แต่มีหลักการจัดสรรหน่วยความจำที่ต่างกันโดยสิ้นเชิง:

- ใน `struct` สมาชิกแต่ละตัวจะมีระยะออฟเซตบนหน่วยความจำแยกเป็นของตนเอง โดยขนาดรวม `sizeof(struct) ≥ ∑ sizeof(members)`
- ใน `union` **สมาชิกทุกตัวจะแชร์จุดเริ่มต้นบนหน่วยความจำตำแหน่งเดียวกัน (Offset 0)** โดยขนาดหน่วยความจำที่จัดสรรจะเท่ากับขนาดของสมาชิกตัวที่ใหญ่ที่สุดเท่านั้น: `sizeof(union) = max(sizeof(members))`

คุณลักษณะ	สตรักต์ (`struct`)	ยูเนียน (`union`)
การจัดสรรหน่วยความจำ	ผลรวมของสมาชิกทุกตัว + ไบต์แพดดิง	เท่ากับขนาดของสมาชิกตัวที่ใหญ่ที่สุด
ตำแหน่งออฟเซตสมาชิก	แยกอิสระ เรียงต่อกันเป็นลำดับ	สมาชิกทุกตัวเริ่มต้นที่ออฟเซต 0 ร่วมกัน
การคงอยู่ของค่าข้อมูล	สมาชิกทุกตัวเก็บค่าอยู่พร้อมกันได้ตลอดเวลา	มีสมาชิกเพียงตัวเดียวเท่านั้นที่เก็บค่าที่ถูกต้องได้ในขณะใดขณะหนึ่ง
กรณีการใช้งานหลัก	การรวบรวมเรกคอร์ดข้อมูล (เช่น Student, Point)	ชนิดข้อมูลแบบ Variant, การแปลงข้อมูลแบบ Type Punning, รีจิสเตอร์ฮาร์ดแวร์

```c
#include <stdio.h>

typedef union {
    int as_int;
    float as_float;
    char as_bytes[4];
} HardwareWord;

int main(void) {
    HardwareWord hw;
    hw.as_int = 0x12345678;

    printf("Union size: %zu bytes\n", sizeof(hw)); /* มีขนาดเพียง 4 ไบต์ */
    printf("Byte 0 (lowest address): 0x%02X\n", (unsigned char)hw.as_bytes[0]);
    printf("Byte 3 (highest address): 0x%02X\n", (unsigned char)hw.as_bytes[3]);
    return 0;
}
```

12.2 ตัวดำเนินการบิตไวส์และการจัดการข้อมูลระดับบิต

ภาษา C มีตัวดำเนินการระดับบิต 6 ตัวที่ทำงานกับรูปแบบบิตไบนารีของจำนวนเต็มโดยตรง:

ตัวดำเนินการ	ชื่อการดำเนินการ	เงื่อนไขทางตรรกะ	การประยุกต์ใช้งานจริง
`&`	บิตไวส์ AND	บิตเป็น 1 ก็ต่อเมื่อบิตทั้งสองฝั่งเป็น 1	การมาสก์บิต (Masking) / การทดสอบบิตเฉพาะตำแหน่ง
`|`	บิตไวส์ OR	บิตเป็น 1 หากมีบิตใดบิตหนึ่งเป็น 1	การเซ็ตเปิดบิต (Turning Bit ON)
`^`	บิตไวส์ XOR	บิตเป็น 1 หากบิตทั้งสองต่างกัน	การสลับสถานะบิต (Toggling Bit)
`~`	บิตไวส์ NOT	กลับบิตทั้งหมดจาก 0 เป็น 1 และ 1 เป็น 0 (เอกภาค)	การสร้างบิตมาสก์ส่วนกลับ
`<<`	เลื่อนบิตไปทางซ้าย	เลื่อนบิตไปทางซ้าย *n* ตำแหน่ง ($x \times 2^n$)	การคูณความเร็วสูง, การสร้างมาสก์
`>>`	เลื่อนบิตไปทางขวา	เลื่อนบิตไปทางขวา *n* ตำแหน่ง ($x / 2^n$)	การหารความเร็วสูง, การดึงฟิลด์บิต

4 สำนวนทองคำของการจัดการบิต (Bitwise Idioms)

```c
unsigned char reg = 0x00; /* 0000 0000 */

/* 1. สั่งเปิดบิต n (Set bit n ให้เป็น 1) */
reg |= (1U << 3);  /* เปิดบิต 3: reg กลายเป็น 0000 1000 */

/* 2. สั่งปิดบิต n (Clear bit n ให้เป็น 0) */
reg &= ~(1U << 3); /* ล้างบิต 3: reg กลับเป็น 0000 0000 */

/* 3. สลับสถานะบิต n (Toggle bit n) */
reg ^= (1U << 2);  /* สลับบิต 2: reg กลายเป็น 0000 0100 */

/* 4. ตรวจสอบสถานะบิต n (Test bit n ว่าเปิดอยู่หรือไม่) */
if (reg & (1U << 2)) {
    printf("Bit 2 is currently ACTIVE\n");
}
```

บทที่ 13

การจัดการไฟล์, สตรีม I/O & การทำงานกับไฟล์ไบนารี

13.1 นามธรรมของสตรีมในภาษา C และโหมดการเปิดไฟล์

การจัดการไฟล์ในภาษา C กระทำผ่านตัวชี้โครงสร้างควบคุมสตรีมที่กำหนดไว้ใน `<stdio.h>` คือ `FILE *` สตรีมไฟล์จะทำหน้าที่ห่อหุ้ม File Descriptor ของระบบปฏิบัติการ พอยน์เตอร์บัฟเฟอร์ภายใน ตำแหน่งมาร์กเกอร์ในไฟล์ และแฟล็กตรวจจับข้อผิดพลาด/EOF

โหมด	คำอธิบาย	หากไฟล์มีอยู่เดิม	หากไฟล์ยังไม่มีอยู่เดิม
`"r"`	เปิดอ่านข้อความ (Read)	เริ่มอ่านจากต้นไฟล์	คืนค่า `NULL` (เกิดข้อผิดพลาด)
`"w"`	เปิดเขียนข้อความ (Write)	**ตัดขนาดเหลือ 0 ไบต์ (ลบเขียนทับ!)**	สร้างไฟล์ว่างชุดใหม่
`"a"`	เปิดเขียนต่อท้าย (Append)	เขียนข้อมูลต่อท้ายไฟล์เดิม	สร้างไฟล์ว่างชุดใหม่
`"r+"`	เปิดทั้งอ่านและเขียน	เริ่มที่ต้นไฟล์	คืนค่า `NULL` (เกิดข้อผิดพลาด)
`"w+"`	เปิดทั้งอ่านและเขียน	ตัดขนาดเหลือ 0 ไบต์	สร้างไฟล์ว่างชุดใหม่
`"rb"` / `"wb"`	อ่าน/เขียนไฟล์ไบนารี	บันทึกข้อมูลไบต์ดิบตรงเป๊ะ ไม่แปลงรหัสขึ้นบรรทัดใหม่	พฤติกรรมมาตรฐาน

13.2 การประมวลผลสตรีมข้อความแบบจัดรูปแบบและอักขระ

API สตรีมมาตรฐานประกอบด้วยการเขียนอ่านแบบจัดรูปแบบ (`fprintf` , `fscanf`) และการจัดการแบบรายบรรทัด (`fgets` , `fputs`):

```c
#include <stdio.h>

int main(void) {
    FILE *fp = fopen("config.txt", "w");
    if (fp == NULL) {
        perror("Failed to open config.txt");
        return 1;
    }

    fprintf(fp, "PORT=%d\nMAX_CONNECTIONS=%d\n", 8080, 250);
    fclose(fp); /* ฟลัชบัฟเฟอร์ของระบบปฏิบัติการและคืนแฮนเดิลไฟล์ */
    return 0;
}
```

13.3 ไฟล์ไบนารี I/O และการเข้าถึงข้อมูลแบบสุ่มตำแหน่ง (Random Access)

ไฟล์ข้อความจะจัดเก็บข้อมูลในรูปของสตริง ASCII/UTF-8 แต่สำหรับ **Binary I/O** จะเป็นการอ่านและเขียนโครงสร้างข้อมูลในหน่วยความจำดิบ ๆ ลงสู่ดิสก์โดยตรงโดยไม่มีการแปลงรูปแบบ ทำให้มีความเร็วสูงสุดผ่านฟังก์ชัน `fread` และ `fwrite` :

- `size_t fwrite(const void *ptr, size_t size, size_t count, FILE *stream)`

- `size_t fread(void *ptr, size_t size, size_t count, FILE *stream)`

การเข้าถึงตำแหน่งแบบสุ่มด้วยฟังก์ชัน fseek() และ ftell()

แทนที่จะต้องอ่านเรียงลำดับจากต้นไฟล์จนจบ ภาษา C รองรับการกระโดดไปยังตำแหน่งใดก็ได้ในไฟล์อย่างรวดเร็ว:

- `int fseek(FILE *stream, long offset, int whence)` :
  - `SEEK_SET` : ออฟเซตเทียบจากจุดเริ่มต้นของไฟล์
  - `SEEK_CUR` : ออฟเซตเทียบจากตำแหน่งปัจจุบันของพอยน์เตอร์ไฟล์
  - `SEEK_END` : ออฟเซตเทียบจากจุดสิ้นสุดของไฟล์
- `long ftell(FILE *stream)` : คืนค่าตำแหน่งออฟเซตปัจจุบันของไฟล์เป็นจำนวนไบต์
- `void rewind(FILE *stream)` : รีเซ็ตพอยน์เตอร์ไฟล์กลับไปที่ออฟเซต 0 ที่ต้นไฟล์

```c
#include <stdio.h>

typedef struct { int id; double score; } Record;

int main(void) {
    Record batch[3] = { {101, 88.5}, {102, 92.0}, {103, 79.5} };
    FILE *fp = fopen("records.bin", "wb+");
    if (!fp) return 1;

    /* เขียนอาร์เรย์ของสตรักต์ลงไฟล์ไบนารีในบล็อกเดียวอย่างรวดเร็ว */
    fwrite(batch, sizeof(Record), 3, fp);

    /* การเข้าถึงแบบสุ่ม: กระโดดตรงไปยังเรกคอร์ดดัชนี 1 โดยไม่ต้องอ่านเรกคอร์ด 0 ก่อน! */
    Record target;
    fseek(fp, 1 * sizeof(Record), SEEK_SET);
    fread(&target, sizeof(Record), 1, fp);

    printf("Directly retrieved Record index 1: ID=%d, Score=%.1f\n", target.id,
target.score);
    fclose(fp);
    return 0;
}
```

ผลลัพธ์การรันคำสั่ง (EXECUTION OUTPUT)

```
Directly retrieved Record index 1: ID=102, Score=92.0
```

บทที่ 14

การจัดสรรหน่วยความจำพลวัตบนฮีป & ความปลอดภัย

14.1 สถาปัตยกรรมฮีปเทียบกับการจัดสรรบนสแตก

การจัดสรรหน่วยความจำบนสแตกมีความรวดเร็ว อัตโนมัติ และแน่นอน แต่มีข้อจำกัดตายตัว: อายุการคงอยู่ของตัวแปรผูกติดกับ Stack Frame ของฟังก์ชัน และขนาดอาร์เรย์จะต้องทราบล่วงหน้าตอนคอมไพล์หรือถูกจำกัดด้วยโควตาของสแตก (เพื่อป้องกันปัญหาสแตกล้น Stack Overflow)

หน่วยความจำฮีป (Heap) เป็นแหล่งจัดสรรหน่วยความจำขนาดใหญ่ที่จัดการได้แบบพลวัต (Dynamic) ขณะรันไทม์ผ่าน API ของ `<stdlib.h>` โดยหน่วยความจำที่ขอจัดสรรบนฮีปจะคงอยู่ตลอดไปจนกว่าโปรแกรมจะสั่งคืนให้กับระบบปฏิบัติการอย่างชัดเจน

ซิกเนเจอร์ของฟังก์ชัน	พฤติกรรมการกำหนดค่าเริ่มต้น	จุดประสงค์การใช้งานทั่วไป	เงื่อนไขเมื่อล้มเหลว
`void* malloc(size_t size)`	**ไม่ได้กำหนดค่าเริ่มต้น** (มีบิตขยะสุ่ม)	การขอหน่วยความจำดิบทั่วไป	คืนค่า `NULL` หากแรมไม่พอ
`void* calloc(size_t num, size_t size)`	**เติมค่าศูนย์ทุกไบต์ (Zero-Initialized)**	การจัดสรรอาร์เรย์ ป้องกันปัญหาโอเวอร์โฟลว์จากการคูณขนาด	คืนค่า `NULL` หากแรมไม่พอ
`void* realloc(void *ptr, size_t new_size)`	รักษาข้อมูลเดิมไว้; ส่วนที่ขยายเพิ่มยังไม่ได้กำหนดค่า	การปรับขนาดบล็อกหน่วยความจำเดิมที่เคยจัดสรรไว้	คืนค่า `NULL` (บล็อกเดิมยังคงอยู่ไม่สูญหาย!)
`void free(void *ptr)`	คืนหน่วยความจำกลับสู่ระบบ	การปลดปล่อยหน่วยความจำฮีปคืนแก่ระบบปฏิบัติการ	ไม่ทำอะไรหาก `ptr == NULL`

14.2 5 กับดักข้อผิดพลาดวิกฤตของหน่วยความจำในภาษา C

เนื่องจากภาษา C ไม่มีระบบเก็บขยะ (Garbage Collector) อัตโนมัติ ผู้พัฒนาจึงต้องรับผิดชอบวงจรชีวิตของหน่วยความจำ 100%:

1. **หน่วยความจำรั่วไหล (Memory Leak)**: การลืมเรียกคำสั่ง `free()` ก่อนที่จะสูญเสียพอยน์เตอร์ตัวสุดท้ายที่ชี้ไปยังบล็อกหน่วยความจำนั้น โปรเซสที่ทำงานต่อเนื่องยาวนานจะกลืนกินแรมของระบบจนหมดสิ้น
2. **พอยน์เตอร์ค้าง (Dangling Pointer)**: การยังคงดีรีเฟอเรนซ์พอยน์เตอร์ต่อไปหลังจากที่บล็อกหน่วยความจำนั้นถูกสั่ง `free()` คืนไปแล้ว
3. **การคืนหน่วยความจำซ้ำซ้อน (Double Free)**: การเรียกคำสั่ง `free()` สองครั้งกับแอดเดรสพอยน์เตอร์ตัวเดิม ซึ่งจะทำลายตาราง Free-List ภายในของตัวจัดการหน่วยความจำฮีป
4. **การดีรีเฟอเรนซ์พอยน์เตอร์ว่าง (Dereferencing NULL)**: การละเลยการตรวจสอบว่า `malloc` คืนค่า `NULL` หรือไม่ก่อนที่จะเขียนข้อมูลลงไป (ทำให้โปรแกรมล่มทันที)
5. **ฮีปบัฟเฟอร์โอเวอร์โฟลว์ (Heap Buffer Overflow)**: การเขียนข้อมูลเกินขอบเขตของบล็อกฮีปที่ขอไว้ ซึ่งจะไปเขียนทับข้อมูลเมทาดาตาของตัวจัดการหน่วยความจำ

14.3 การอิมพลีเมนต์โครงสร้างข้อมูลเวกเตอร์แบบพลวัต (Dynamic Vector)

ตัวอย่างระดับโปรดักชันต่อไปนี้สาธิตการจัดสรรหน่วยความจำ การขยายขนาดอย่างปลอดภัยด้วยพอยน์เตอร์ชั่วคราวเพื่อป้องกันเมมโมรีลีค และการล้างข้อมูลอย่างปลอดภัย:

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct {
    int *data;
    size_t size;
    size_t capacity;
} IntVector;

IntVector* vector_create(size_t initial_capacity) {
    IntVector *vec = (IntVector*)malloc(sizeof(IntVector));
    if (!vec) return NULL;

    vec->data = (int*)malloc(initial_capacity * sizeof(int));
    if (!vec->data) {
        free(vec);
        return NULL;
    }
    vec->size = 0;
    vec->capacity = initial_capacity;
    return vec;
}

int vector_push(IntVector *vec, int val) {
    if (vec->size >= vec->capacity) {
        size_t new_cap = vec->capacity * 2;
        /* ใช้พอยน์เตอร์ชั่วคราวรับค่า realloc เสมอ เพื่อป้องกัน memory leak หากล้มเหลว! */
        int *tmp = (int*)realloc(vec->data, new_cap * sizeof(int));
        if (!tmp) return 0; /* การขยายขนาดล้มเหลว แต่ข้อมูลเดิมยังคงปลอดภัยไม่สูญหาย */
        vec->data = tmp;
        vec->capacity = new_cap;
    }
    vec->data[vec->size++] = val;
    return 1;
}

void vector_destroy(IntVector **vec_ptr) {
    if (vec_ptr && *vec_ptr) {
        free((*vec_ptr)->data);
        free(*vec_ptr);
        *vec_ptr = NULL; /* รูปแบบเชิงป้องกัน: รีเซ็ตพอยน์เตอร์ให้เป็น NULL */
    }
}

int main(void) {
    IntVector *v = vector_create(2);
    for (int i = 1; i <= 5; i++) vector_push(v, i * 10);

    printf("Vector size: %zu | Capacity: %zu\nValues: ", v->size, v->capacity);
    for (size_t i = 0; i < v->size; i++) printf("%d ", v->data[i]);
    printf("\n");

    vector_destroy(&v);
```

```
    return 0;
}
```

ผลลัพธ์การรันคำสั่ง (EXECUTION OUTPUT)

```
Vector size: 5 | Capacity: 8
Values: 10 20 30 40 50
```

บทที่ 15

พรีโปรเซสเซอร์, สุขอนามัยของมาโคร & สถาปัตยกรรมเฮดเดอร์

15.1 การขยายคำสั่งมาโครและกฎเหล็กการใส่วงเล็บป้องกันข้อผิดพลาด

พรีโปรเซสเซอร์ของภาษา C ทำงานโดยการแทนที่ข้อความดิบล้วน ๆ ก่อนการคอมไพล์ มาโครที่มีลักษณะคล้ายฟังก์ชันแต่ไม่ได้ใส่วงเล็บครอบพารามิเตอร์แต่ละตัวและนิพจน์ทั้งหมดอย่างเคร่งครัด จะก่อให้เกิดบั๊กลำดับความสำคัญที่ร้ายแรง:

```
/* มาโครอันตรายที่มีบั๊ก: */
#define BAD_SQUARE(x) x * x
/* การเรียกใช้งาน: BAD_SQUARE(2 + 3) จะถูกแทนที่ข้อความเป็น: 2 + 3 * 2 + 3 == 2 + 6 + 3 == 11 (ไม่ใช่ 25!)
*/

/* มาโครที่ถูกสุขอนามัย: ทุกพารามิเตอร์และทั้งนิพจน์ถูกครอบด้วยวงเล็บอย่างรัดกุม */
#define SAFE_SQUARE(x) ((x) * (x))
/* การเรียกใช้งาน: SAFE_SQUARE(2 + 3) จะแทนที่เป็น: ((2 + 3) * (2 + 3)) == (5 * 5) == 25 */
```

15.2 ตัวดำเนินการพรีโปรเซสเซอร์ขั้นสูง: # และ

ภาษา C มาตรฐานมีตัวดำเนินการเขียนโปรแกรมระดับเมทาที่ทรงพลัง 2 ตัว:

- **ตัวดำเนินการแปลงเป็นสตริง (Stringification: `#`)**: แปลงอาร์กิวเมนต์ของมาโครให้กลายเป็นสตริงลิเทอรัลในเครื่องหมายคำพูด
- **ตัวดำเนินการเชื่อมต่อโทเค็น (Token Concatenation: `##`)**: ผสานสองโทเค็นทางไวยากรณ์เข้าด้วยกันเพื่อสร้างเป็นตัวระบุชื่อใหม่ตัวเดียว

```
#include <stdio.h>

/* การแปลงเป็นสตริง: พิมพ์ข้อความนิพจน์คู่กับค่าผลลัพธ์ที่คำนวณได้ */
#define PRINT_EXPR(expr) printf(#expr " = %d\n", (expr))

/* การเชื่อมโทเค็น: สร้างฟังก์ชัน Getter เฉพาะชนิดข้อมูลแบบอัตโนมัติ */
#define DECLARE_GETTER(type, name) \
    type get_##name(void) { return current_##name; }

int current_temperature = 24;
DECLARE_GETTER(int, temperature) /* สร้างฟังก์ชัน: int get_temperature(void) ... */

int main(void) {
    PRINT_EXPR(10 + 20 * 3);
    printf("Temperature via generated getter: %d\n", get_temperature());
    return 0;
}
```

15.3 การออกแบบไฟล์เฮดเดอร์: อินคลูดการ์ด (Include Guards) และความเป็นโมดูล

เมื่อมีไฟล์ `.c` หลายไฟล์นำเข้าเฮดเดอร์ตัวเดียวกัน จะเกิดข้อผิดพลาดประกาศชนิดข้อมูลซ้ำซ้อนตอนคอมไพล์ เว้นแต่จะป้องกันไว้ด้วย **Include Guards**:

```c
/* ภายในไฟล์: math_utils.h */
#ifndef MATH_UTILS_H
#define MATH_UTILS_H

/* ประกาศโพรโทไทป์ของฟังก์ชัน */
double vector_magnitude(double x, double y);

#endif /* MATH_UTILS_H */
```

บทที่ 16

อาร์กิวเมนต์ CLI, ไลบรารีมาตรฐาน & การดักจับข้อผิดพลาด

16.1 อาร์กิวเมนต์จากบรรทัดคำสั่ง: argc และ argv

โปรแกรมสามารถรับพารามิเตอร์การประมวลผลผ่านทางบรรทัดคำสั่งได้ผ่าน 2 พารามิเตอร์ในฟังก์ชัน `main` :

- `int argc` : จำนวนอาร์กิวเมนต์ที่ส่งเข้ามา (นับรวมชื่อโปรแกรมไบนารีเองด้วย)
- `char *argv[]` : อาร์เรย์ของพอยน์เตอร์สตริง โดย `argv[0]` คือชื่อเส้นทางโปรแกรมเสมอ และ `argv[argc]` รับประกันว่ามีค่าเป็น `NULL`

16.2 การแปลงข้อมูลที่รัดกุมผ่านฟังก์ชัน strtol()

ฟังก์ชันยุคเก่าอย่าง `atoi()` ไม่สามารถตรวจสอบอักขระที่ไม่ใช่ตัวเลขได้ และจะคืนค่า `0` ทั้งในกรณีที่เกิดข้อผิดพลาดและกรณีที่ผู้ใช้ป้อนเลข 0 จริง ๆ ฟังก์ชันมาตรฐานระดับอุตสาหกรรมที่แข็งแกร่งคือ **`strtol()`** :

```c
#include <stdio.h>
#include <stdlib.h>
#include <errno.h>

int main(int argc, char *argv[]) {
    if (argc < 2) {
        fprintf(stderr, "Usage: %s <number>\n", argv[0]);
        return 1;
    }

    char *endptr;
    errno = 0;
    long val = strtol(argv[1], &endptr, 10);

    if (errno != 0 || *endptr != '\0') {
        fprintf(stderr, "Error: %s is not a valid integer.\n", argv[1]);
        return 1;
    }

    printf("Successfully parsed number: %ld\n", val);
    return 0;
}
```

16.3 การวินิจฉัยข้อผิดพลาดของระบบ: errno, perror() & assert()

ฟังก์ชันในไลบรารีมาตรฐานจะส่งสัญญาณข้อผิดพลาดโดยการกำหนดค่าให้กับตัวแปรโกลบอล **`errno`** (จาก `<errno.h>`):

- `void perror(const char *prefix)` : พิมพ์ข้อความนำทางที่กำหนดตามด้วยคำอธิบายข้อผิดพลาดของระบบที่ตรงกับค่า `errno` ปัจจุบัน
- `char *strerror(int errnum)` : คืนค่าพอยน์เตอร์สตริงคำอธิบายข้อผิดพลาดที่มนุษย์อ่านเข้าใจได้
- `assert(expression)` (จาก `<assert.h>`): ประเมินค่านิพจน์ระหว่างขั้นตอนการพัฒนา หากเป็นเท็จจะสั่งยุติโปรแกรมทันทีพร้อมระบุชื่อไฟล์ บรรทัด และนิพจน์ที่ผิดพลาด โดยสามารถปิดการทำงานในโปรดักชันได้ด้วยการใส่ `#define NDEBUG`

บทที่ 17

พฤติกรรมที่ไม่พึงประสงค์ (UB), การเขียนโค้ดเชิงป้องกัน & กับดักข้อผิดพลาด

17.1 ไตรภาคพฤติกรรมตามมาตรฐานภาษา C

มาตรฐาน ISO C จัดแบ่งพฤติกรรมของโปรแกรมออกเป็น 3 ระดับสำคัญ:

1. **พฤติกรรมที่ขึ้นกับการอิมพลีเมนต์ (Implementation-Defined Behavior)**: โค้ดที่ถูกต้องตามมาตรฐาน โดยมาตรฐานเปิดโอกาสให้ผู้พัฒนาคอมไพเลอร์เป็นผู้ตัดสินใจพฤติกรรมและต้องระบุไว้ในเอกสารคู่มือ (เช่น `char` เป็นแบบมีเครื่องหมายหรือไม่มีเครื่องหมาย, ขนาดของ `int`)
2. **พฤติกรรมที่ไม่เจาะจง (Unspecified Behavior)**: มีความเป็นไปได้ 2 ทางหรือมากกว่านั้น โดยคอมไพเลอร์ไม่จำเป็นต้องบันทึกเป็นเอกสาร (เช่น ลำดับการประเมินอาร์กิวเมนต์ของฟังก์ชัน: ใน `func(a(), b())` ฟังก์ชัน `a()` หรือ `b()` อาจถูกเรียกก่อนก็ได้)
3. **พฤติกรรมที่ไม่พึงประสงค์ (Undefined Behavior: UB)**: ***สภาวะที่อันตราย คาดเดาไม่ได้ และไม่สามารถพกพาข้ามระบบได้*** มาตรฐานไม่ได้กำหนดข้อบังคับใด ๆ ไว้เลย คอมไพเลอร์อาจลบโค้ดส่วนนั้นทิ้ง สร้างการคำนวณที่ผิดพลาด โปรแกรมแครชด้วย Segfault หรืออาจดูเหมือนทำงานได้ชั่วคราว

17.2 แกลเลอรีรวบรวมกับดักอันตรายที่พบบ่อยในภาษา C

กับดักข้อผิดพลาดอันตราย	ตัวอย่างโค้ดที่มีความเสี่ยง	สาเหตุและผลกระทบ	แนวทางแก้ไขเชิงป้องกันที่แข็งแกร่ง
การพยายามแก้ไขลิเทอรัลสตริง	`char *s = "hello"; s[0]=H;`	ลิเทอรัลสตริงอยู่ในหน่วยความจำอ่านอย่างเดียว (`.rodata`) ส่งผลให้เกิด Segfault ทันที	ใช้อาร์เรย์ที่แก้ไขได้: `char s[] = "hello";`
การเข้าถึงดัชนีเกินขอบเขตไปหนึ่ง (Off-By-One)	`for(int i=0; i<=N; i++) a[i]=0;`	เข้าถึงตำแหน่ง `a[N]` ซึ่งเขียนทับตัวแปรอื่นบนสแตก	ใช้เครื่องหมายน้อยกว่าอย่างเคร่งครัด: `i < N`
การคืนค่าพอยน์เตอร์ตัวแปรบนสแตก (Dangling Stack Pointer)	`int* f() { int x=5; return &x; }`	คืนแอดเดรสของ Stack Frame ที่ถูกทำลายทิ้งไปแล้วทันทีหลังฟังก์ชันทำงานจบ	จัดสรรบนฮีปด้วย `malloc` หรือส่งพอยน์เตอร์รับค่าเข้ามา
การไม่ตรวจผลลัพธ์ของคำสั่ง scanf	`scanf("%d", &val);`	หากผู้ใช้ป้อนอักขระที่ไม่ใช่ตัวเลข `scanf` จะล้มเหลวเงียบ ๆ และ `val` จะเก็บค่าขยะ	ตรวจสอบค่าคืนกลับเสมอ: `if (scanf(...) == 1)`
การโอเวอร์โฟลว์ของจำนวนเต็มมีเครื่องหมาย	`int z = INT_MAX + 1;`	Signed Integer Overflow ถือเป็น Undefined Behavior ในภาษา C	ตรวจสอบขอบเขตก่อนบวก หรือใช้ชนิดข้อมูลแบบ unsigned

กฎสุขอนามัยของคอมไพเลอร์: ปฏิบัติต่อคำเตือนเสมือนเป็นข้อผิดพลาดร้ายแรง

ควรคอมไพล์โครงงานภาษา C ทุกครั้งโดยเปิดแฟล็กแจ้งเตือนอย่างเข้มงวดสูงสุด:
`gcc -Wall -Wextra -Wpedantic -Werror -std=c11 main.c -o program`
คอมไพเลอร์คือเครื่องมือตรวจสอบเชิงป้องกันด่านแรกที่มีประสิทธิภาพสูงสุดของคุณ

ภาคผนวก A

ตารางลำดับความสำคัญและการจัดกลุ่มตัวดำเนินการ 15 ระดับ

ในภาษา C ลำดับความสำคัญและทิศทางการจัดกลุ่มของตัวดำเนินการ (Precedence & Associativity) เป็นตัวกำหนดลำดับการประมวลผลนิพจน์ที่ไม่มีการใส่วงเล็บ โดยตัวดำเนินการระดับ 1 จะมีอำนาจผูกมัดสูงสุด และระดับ 15 จะมีอำนาจผูกมัดต่ำสุด

ระดับ	ตัวดำเนินการ	คำอธิบาย	ทิศทางการจัดกลุ่ม (Associativity)
1 (สูงสุด)	`()` `[]` `->` `.` `++` `--`	การเรียกใช้ฟังก์ชัน, ดัชนีอาร์เรย์, ตัวดำเนินการลูกศร พอยน์เตอร์, จุดสตรักต์, Postfix เพิ่ม/ลดค่า	จากซ้ายไปขวา
2	`++` `--` `+` `-` `!` `~` `(type)` `*` `&` `sizeof`	Prefix เพิ่ม/ลดค่า, เครื่องหมายบวก/ลบเอกภาค, NOT ตรรกะ, NOT บิตไวส์, Type cast, ดีรีเฟอเรนซ์, แอดเดรส, ขนาดข้อมูล	**จากขวาไปซ้าย**
3	`*` `/` `%`	การคูณ, การหาร, มอดุโลเศษ	จากซ้ายไปขวา
4	`+` `-`	การบวก, การลบ	จากซ้ายไปขวา
5	`<<` `>>`	บิตไวส์เลื่อนบิตไปทางซ้าย, เลื่อนบิตไปทางขวา	จากซ้ายไปขวา
6	`<` `<=` `>` `>=`	การเปรียบเทียบ น้อยกว่า, น้อยกว่าเท่ากับ, มากกว่า, มากกว่าเท่ากับ	จากซ้ายไปขวา
7	`==` `!=`	ความเท่ากัน: เท่ากับ, ไม่เท่ากับ	จากซ้ายไปขวา
8	`&`	บิตไวส์ AND	จากซ้ายไปขวา
9	`^`	บิตไวส์ XOR (Exclusive OR)	จากซ้ายไปขวา
10	`|`	บิตไวส์ OR (Inclusive OR)	จากซ้ายไปขวา
11	`&&`	ตรรกะ AND (รับประกันการประเมินแบบลัดวงจร)	จากซ้ายไปขวา
12	`||`	ตรรกะ OR (รับประกันการประเมินแบบลัดวงจร)	จากซ้ายไปขวา
13	`? :`	ตัวดำเนินการเงื่อนไขแบบ Ternary	**จากขวาไปซ้าย**
14	`=` `+=` `-=` `*=` `/=` `%=` `<<=` `>>=` `&=` `^=` `|=`	การกำหนดค่าโดยตรงและการกำหนดค่าแบบประกอบเลขคณิต/บิตไวส์	**จากขวาไปซ้าย**
15 (ต่ำสุด)	`,`	ตัวดำเนินการจุลภาค (Comma Operator: ประเมินตามลำดับแล้วคืนค่าขวาสุด)	จากซ้ายไปขวา

ภาคผนวก B

ตารางอ้างอิงรหัสจัดรูปแบบข้อมูลมาตรฐาน

รหัสระบุการแปลงข้อมูล (Conversion Specifiers) ทำหน้าที่ควบคุมฟังก์ชัน I/O แบบจัดรูปแบบ (`printf` , `scanf` , `fprintf` , `sprintf`) ในการตีความและแสดงผลไบต์:

รหัสระบุ	ชนิดข้อมูลภาษา C ที่รองรับ	รูปแบบและคุณลักษณะการแสดงผล	ข้อควรระวังใน scanf
`%d` / `%i`	`int`	จำนวนเต็มฐานสิบมีเครื่องหมาย	ต้องการ `int*`
`%u`	`unsigned int`	จำนวนเต็มฐานสิบไม่มีเครื่องหมาย	ต้องการ `unsigned int*`
`%ld`	`long int`	จำนวนเต็มยาว 64 บิตมีเครื่องหมาย	ต้องการ `long*`
`%lu`	`unsigned long`	จำนวนเต็มยาวไม่มีเครื่องหมาย	ต้องการ `unsigned long*`
`%f`	`float` / `double`	ทศนิยม (เช่น `3.141593`)	ใน `scanf` `%f` คือ `float*` แต่ **`%lf` จำเป็นสำหรับ `double*` !**
`%lf`	`double`	ทศนิยมความละเอียดคู่	บังคับใช้สำหรับ `scanf("%lf", &dbl)`
`%e` / `%E`	`double`	สัญกรณ์วิทยาศาสตร์ (เช่น `1.234500e+03`)	ขนาดตัวอักษรระบุสัญลักษณ์เลขชี้กำลัง
`%g` / `%G`	`double`	รูปแบบกะทัดรัด (เลือกระหว่าง `%f` หรือ `%e` อัตโนมัติ)	ตัดเลขศูนย์ส่วนเกินท้ายทิ้ง
`%c`	`char`	อักขระเดี่ยว	**ไม่ข้ามช่องว่างนำหน้าใน scanf!**
`%s`	`char*` / `char[]`	ลำดับอักขระที่ไม่ใช่ช่องว่าง	หยุดอ่านเมื่อเจอเว้นวรรค/ขึ้นบรรทัดใหม่; ไม่ต้องใส่ `&`
`%p`	`void*`	แอดเดรสพอยน์เตอร์เลขฐานสิบหก (เช่น `0x7ffeefbff5ac`)	ควรแคสต์อาร์กิวเมนต์เป็น `(void*)` ใน `printf`
`%x` / `%X`	`unsigned int`	เลขฐานสิบหก (ตัวพิมพ์เล็ก / ตัวพิมพ์ใหญ่)	แสดงผลโดยไม่มี `0x` นำหน้า
`%o`	`unsigned int`	เลขฐานแปด (Base 8)	แสดงผลโดยไม่มี `0` นำหน้า
`%%`	ไม่มี	สัญลักษณ์เครื่องหมายเปอร์เซ็นต์ (`%`)	ไม่ใช้อาร์กิวเมนต์
`%zu`	`size_t`	จำนวนเต็มบวกแสดงขนาด (จาก `sizeof`)	รหัสมาตรฐาน C99

ภาคผนวก C

รหัสหลีกมาตรฐาน & เส้นทางวิวัฒนาการของมาตรฐานภาษา C

เส้นทางวิวัฒนาการของมาตรฐานภาษา C

- **1972 - Bell Labs C**: สร้างขึ้นโดย Dennis Ritchie ณ Bell Labs เพื่อใช้ในการพัฒนาระบบปฏิบัติการ UNIX
- **1978 - K&R C**: Brian Kernighan และ Dennis Ritchie ตีพิมพ์หนังสือ *The C Programming Language* (ฉบับพิมพ์ครั้งแรก)
- **1989/1990 - ANSI C / C89 / C90 (ISO/IEC 9899:1990)**: มาตรฐานทางการฉบับแรก กำหนดระบบโพรโทไทป์ของฟังก์ชัน เฮดเดอร์มาตรฐาน และคำสงวน `const` / `volatile`
- **1999 - C99 (ISO/IEC 9899:1999)**: เพิ่มเติม Variable-Length Arrays (VLAs), Designated Initializers, ฟังก์ชันอินไลน์, คอมเมนต์บรรทัดเดียว (`//`), `<stdint.h>` , `<stdbool.h>`
- **2011 - C11 (ISO/IEC 9899:2011)**: เพิ่มการประมวลผลแบบมัลติเธรด (`<threads.h>`), Atomic Operations, `_Generic` , สตรักต์นิรนาม และตัดฟังก์ชัน `gets()` ทิ้งอย่างเป็นทางการ
- **2018 - C17 / C18 (ISO/IEC 9899:2018)**: มาตรฐานปรับปรุงแก้ไขข้อบกพร่องโดยไม่มีการเพิ่มคุณลักษณะภาษาขนาดใหญ่
- **2024 - C23**: มาตรฐานยุคใหม่ เพิ่มคีย์เวิร์ด `nullptr` , `constexpr` , `bool` / `true` / `false` โดยกำเนิด, `typeof` และแอตทริบิวต์มาตรฐาน (เช่น `[[fallthrough]]` , `[[nodiscard]]`)

รหัสหลีกมาตรฐาน (Standard Escape Sequences)

รหัสหลีก	รหัส Hex / ASCII	ชื่อ	คำอธิบายการทำงาน
	`0x0A`	Newline	เลื่อนเคอร์เซอร์ไปยังจุดเริ่มต้นของบรรทัดถัดไป
`\t`	`0x09`	Horizontal Tab	เลื่อนเคอร์เซอร์ไปยังจุดตั้งแท็บถัดไป (โดยทั่วไปคือ 4 หรือ 8 คอลัมน์)
`\\`	`0x5C`	Backslash	แสดงผลอักขระแบ็กสแลช (`\`)
`\'`	`0x27`	Single Quote	แสดงผลอักขระอัญประกาศเดี่ยวในค่าคงที่อักขระ (`'`)
`\"`	`0x22`	Double Quote	แสดงผลอักขระอัญประกาศคู่ในลิเทอรัลสตริง (`"`)
`\0`	`0x00`	Null Character	อักขระว่างสำหรับปิดท้ายสตริงอาร์เรย์มาตรฐานในภาษา C
`\r`	`0x0D`	Carriage Return	เลื่อนเคอร์เซอร์กลับไปที่จุดเริ่มต้นของบรรทัดเดิมโดยไม่เลื่อนลงข้างล่าง
`\b`	`0x08`	Backspace	ถอยหลังเคอร์เซอร์กลับไป 1 ตำแหน่ง
`\a`	`0x07`	Audible Alert (Bell)	ส่งเสียงแจ้งเตือนของระบบหรือกะพริบหน้าจอ